HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG



BÁO CÁO BÀI THỰC HÀNH 1: TÀI LIỆU SRS
HỌC PHẦN: IOT VÀ ỨNG DỤNG

Giảng viên hướng dẫn : TS. Nguyễn Quốc Uy

Họ và tên sinh viên : Lê Thế Hoàng

Mã sinh viên : B23DCCN331

Lớp : D23CQCN09-B

Hà Nội – 2026

Mục Lục

I. CHƯƠNG 1: GIỚI THIỆU CHUNG. 4
1. Tìm hiểu về đề tài 4
2. Phạm vi, đối tượng nghiên cứu 5
3. Kiến trúc và lựa chọn công nghệ 5
II. CHƯƠNG 2: THIẾT KẾ HỆ THỐNG TỔNG QUAN 6
1. Sơ đồ cấu trúc toàn bộ hệ thống. 6
2. Luồng hoạt động của hệ thống. 7
a) Luồng giám sát **7**
b) Luồng điều khiển **7**
c) Luồng truy vấn, lưu trữ dữ liệu **8**
III. CHƯƠNG 3: USECASE VÀ THIẾT KẾ HỆ THỐNG CHI TIẾT. **8**
1. Mô tả Usecase và Sequence diagram **9**
1.1. UC01: Đăng nhập **9**
a) Giao diện **9**
b) Mô tả Usecase 01: Đăng nhập **10**
c) Sơ đồ tuần tự. **11**
d) Mô tả luồng **11**
1.2. UC02: Theo dõi số liệu cảm biến đo ở thời gian thực **12**
a) Giao diện **12**
b) Mô tả Usecase 02: Theo dõi số liệu cảm biến đo ở thời gian thực **12**
c) Sequence diagram. **13**
d) Mô tả luồng **13**
1.3. UC03: Hiển thị biểu đồ dữ liệu cảm biến **14**
a) Giao diện **14**
b) Mô tả Usecase 03: Hiển thị biểu đồ dữ liệu cảm biến **14**
c) Sequence diagram. **15**
d) Mô tả luồng **16**
1.4. UC04: Điều khiển bật/tắt thiết bị **16**
a) Giao diện **16**
b) Mô tả Usecase 04: Điều khiển bật/tắt thiết bị **17**

c) Sequence diagram **18**
d) Mô tả luồng **19**
1.5. UC05: Xem lịch sử cảm biến đo số liệu **21**
a) Giao diện **21**
b) Mô tả Usecase 05: Xem lịch sử cảm biến đo số liệu **21**
c) Sequence diagram. **22**
d) Mô tả luồng **23**
1.6. UC06: Xem lịch sử bật/tắt thiết bị **24**
a) Giao diện **24**
b) Mô tả Usecase 06: Xem lịch sử bật/tắt thiết bị **24**
c) Sequence diagram. **225**
d) Mô tả luồng **236**
1.7. Giao diện trang Profile **27**
2. Database và sơ đồ ERD **28**
2.1. Bảng Users **28**
2.2. Bảng Sensors **28**
2.3. Bảng SensorData **29**
2.4. Bảng Devices **29**
2.5. Bảng Actions **29**
3. API **30**
3.1. API 1: Đăng nhập hệ thống (Authentication) **30**
3.2. Kênh kết nối WebSocket: Đẩy số liệu thời gian thực (Real-time Stream) **31**
3.3. API 2: Lấy dữ liệu phục vụ biểu đồ giám sát (Monitoring Chart) **32**
3.4. API 3: Truy vấn và phân trang lịch sử dữ liệu cảm biến (Sensor Data) **33**
3.5. API 4: Gửi lệnh điều khiển thiết bị (Device Control) **34**
3.6. API 5: Truy vấn nhật ký lịch sử điều khiển (Action History) **36**
3.7. API 6: Lấy danh sách và trạng thái thiết bị **36**
IV. CHƯƠNG 4: XÂY DỰNG VÀ THỰC NGHIỆM HỆ THỐNG **38**
1. Triển khai phần cứng thực tế **40**
2. Triển khai phần mềm và dịch vụ **42**
3. Kết quả thực nghiệm trên giao diện Web **44**

4. Đánh giá và nhận xét hệ thống 46

I. CHƯƠNG 1: GIỚI THIỆU CHUNG.

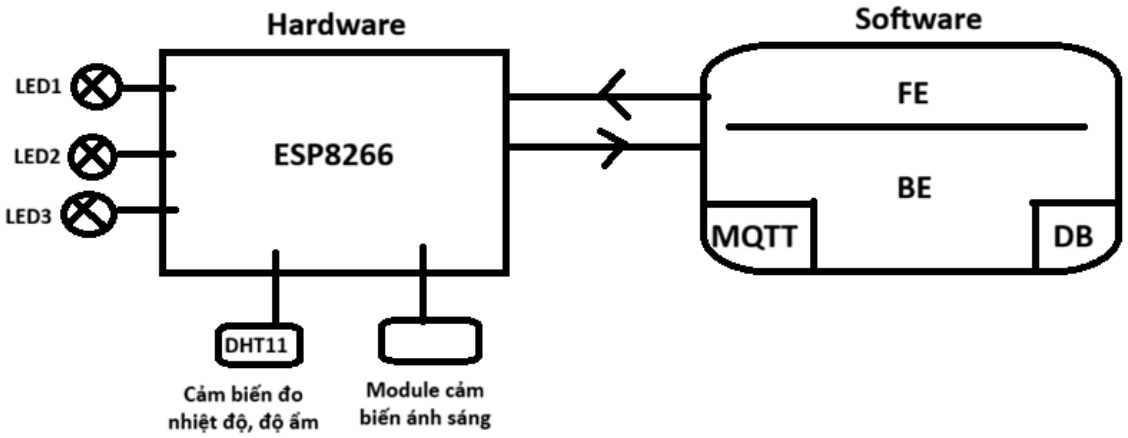
1. Tìm hiểu về đề tài

Đề tài là: "Xây dựng hệ thống IoT giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển thiết bị", được thực hiện với các mục tiêu chính sau:

-Thu thập dữ liệu: Đọc các thông số nhiệt độ, độ ẩm (không khí) và cường độ ánh sáng từ các cảm biến.

-Hiển thị thông số: Truyền dữ liệu thu thập được theo thời gian thực (cập nhật 2 giây 1 lần) và hiển thị trực quan lên giao diện màn hình Web (Web Server Dashboard).

-Điều khiển hệ thống: Người dùng có thể nhấn nút trên giao diện Web để bật/tắt chế độ hoạt động của thiết bị từ xa. Hệ thống sử dụng 3 đèn LED để tượng trưng cho các thiết bị như quạt, điều hòa, đèn điện.



2. Phạm vi, đối tượng nghiên cứu

-Phạm vi nghiên cứu:

Tập trung vào việc thiết kế kiến trúc phần mềm, xây dựng cơ sở dữ liệu, thiết kế giao diện UI/UX, đặc tả hệ thống RESTful API và mô phỏng luồng truyền tin thời gian thực giữa giao diện web, tầng dịch vụ backend và phần cứng IoT. Hệ thống áp dụng cho mô hình giám sát và điều khiển trong môi trường dân dụng và học tập.

-Đối tượng nghiên cứu:

- Các mô hình kết nối mạng IoT dân dụng
- Giao thức truyền tin nhẹ MQTT (Message Queuing Telemetry Transport)
- Mô hình client-server trong quản lý thiết bị thông minh
- Cấu trúc lưu trữ dữ liệu chuỗi thời gian (time-series data) cho các cảm biến môi trường
- Vi điều khiển ESP8266 (NodeMCU) và các cảm biến DHT11, cảm biến ánh sáng.

3. Kiến trúc và lựa chọn công nghệ

Hệ thống được chia thành 3 phần chính:

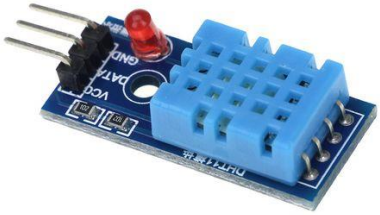
-Phần cứng (Hardware): Sử dụng vi điều khiển ESP8266 (NodeMCU) tích hợp sẵn module WiFi làm bộ não trung tâm. Thiết bị này kết nối với cảm biến DHT11 (đo nhiệt độ, độ ẩm), cảm biến ánh sáng (đo ánh sáng) và 3 đèn LED. Phần cứng được lập trình nhúng bằng ngôn ngữ C++ thông qua phần mềm Arduino IDE.

-Giao thức truyền thông: Hệ thống sử dụng giao thức MQTT để đảm bảo việc truyền

nhận dữ liệu nhanh, nhẹ giữa vi điều khiển và máy chủ Web.

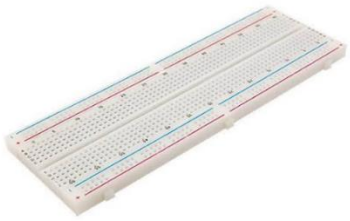
-Phần mềm (Software): Bao gồm Web Server đóng vai trò nhận dữ liệu hiển thị lên Dashboard và một Cơ sở dữ liệu (Database) để lưu trữ lịch sử.





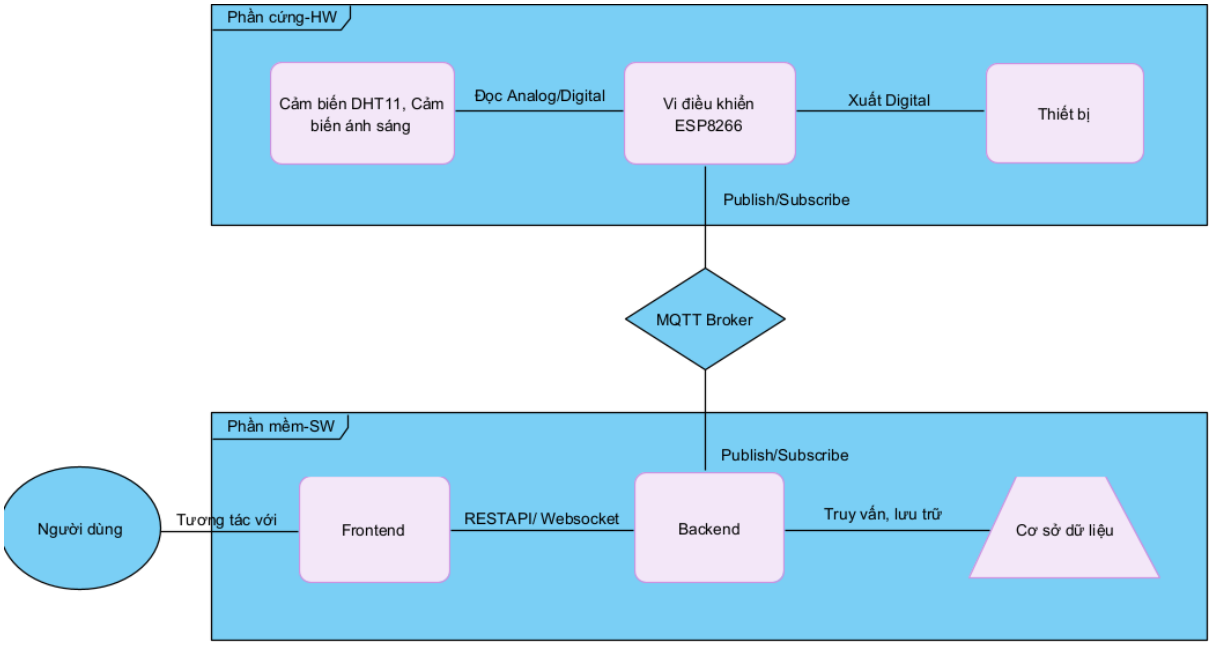






II. CHƯƠNG 2: THIẾT KẾ HỆ THỐNG TỔNG QUAN

1. Sơ đồ cấu trúc toàn bộ hệ thống.



Hình 1. Sơ đồ cấu trúc hệ thống

2. Luồng hoạt động của hệ thống.

a) Luồng giám sát

-Thu thập: Theo chu kỳ 2 giây/lần, ESP8266 đọc giá trị nhiệt độ, độ ẩm từ DHT11 và cường độ ánh sáng từ cảm biến ánh sáng.

-Tiếp nhận & lưu trữ: Backend đã subscribe topic này, kiểm tra tính hợp lệ của dữ liệu rồi lưu một bản ghi mới vào bảng SensorData trong Cơ sở dữ liệu.

-Đóng gói & truyền tin: Dữ liệu được đóng gói dạng JSON và publish lên MQTT Broker (topic dữ liệu cảm biến).

-Cập nhật giao diện thời gian thực: Ngay sau khi lưu thành công, Backend đẩy dữ liệu mới nhất tới Frontend qua kênh WebSocket (/ws/realtime) để cập nhật số liệu và biểu đồ trên Dashboard mà không cần tải lại trang.

b) Luồng điều khiển

-Gửi lệnh: Khi người dùng nhấn nút Bật/Tắt trên Dashboard, Frontend gửi yêu cầu POST /api/devices/{id}/control tới Backend.

-Ghi nhận & phát lệnh: Backend tạo một bản ghi mới trong bảng Actions với trạng thái PENDING, đồng thời publish lệnh điều khiển lên topic MQTT "iot/device/control/{id}".

-Thực thi tại phần cứng: MQTT Broker chuyển tiếp lệnh tới ESP8266 (đã subscribe topic điều khiển); ESP8266 thực hiện bật/tắt đèn LED tương ứng rồi publish trạng thái

thực tế lên topic phản hồi "iot/device/state/{id}".

-Phản hồi & cập nhật: Backend nhận trạng thái thực tế qua MQTT, cập nhật bảng Devices (trạng thái thực tế) và bảng Actions (SUCCESS/FAILED), sau đó đẩy kết quả tới Frontend để cập nhật giao diện.

-Xử lý ngoại lệ: nếu quá thời gian chờ quy định (timeout, ví dụ 5 giây) mà không nhận được phản hồi từ phần cứng, Backend đánh dấu bản ghi Actions là FAILED và Frontend hiển thị cảnh báo "Thiết bị không phản hồi".

c) Luồng truy vấn, lưu trữ dữ liệu

-Yêu cầu từ người dùng: Khi người dùng thiết lập bộ lọc (khoảng thời gian, loại cảm biến, thiết bị, trạng thái) và phân trang trên trang Lịch sử cảm biến hoặc Lịch sử thiết bị, Frontend gửi yêu cầu tới GET /api/sensor-data/history hoặc GET /api/actions/history kèm các tham số tương ứng.

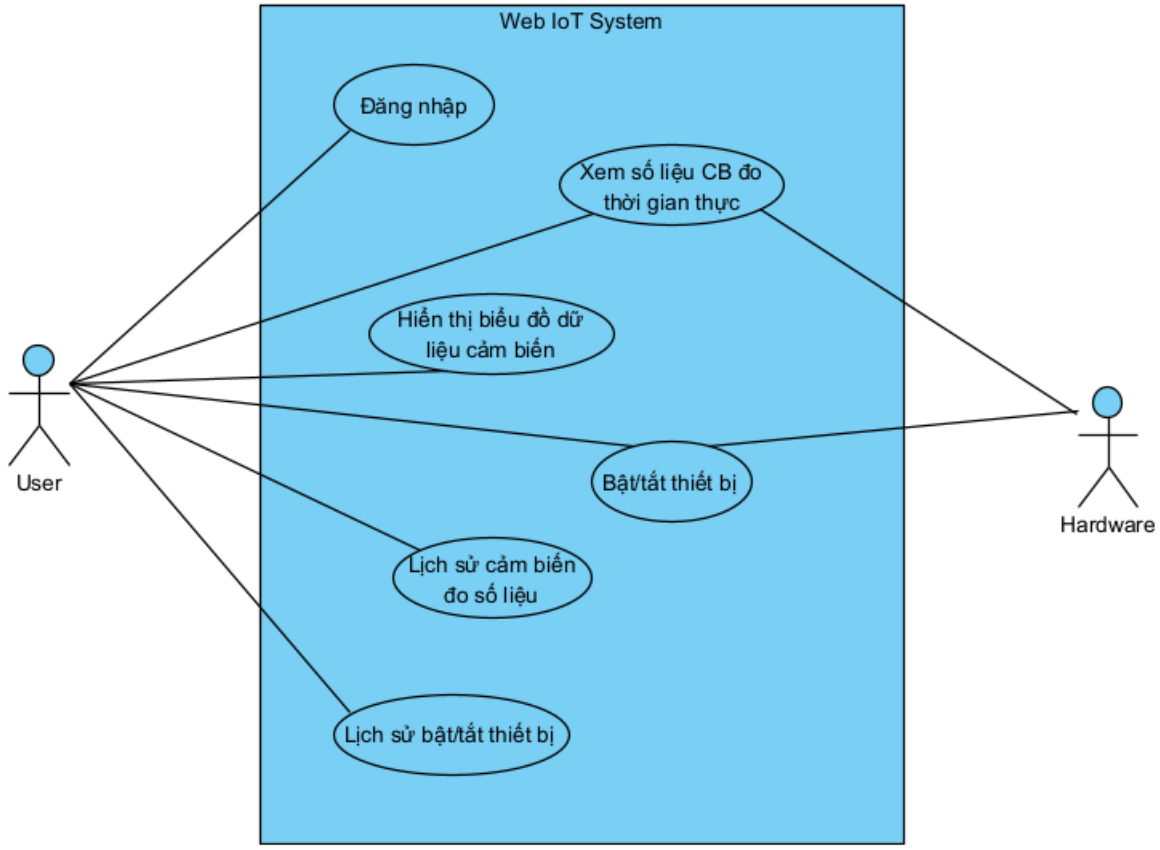
-Xử lý tại Backend: Backend truy vấn bảng SensorData (JOIN Sensors) hoặc bảng Actions (JOIN Devices, Users) theo điều kiện lọc, có phân trang (LIMIT/OFFSET).

-Trả kết quả: Danh sách bản ghi cùng thông tin số trang được đóng gói và trả về cho Frontend để hiển thị lên bảng dữ liệu.

III. CHƯƠNG 3: USECASE VÀ THIẾT KẾ HỆ THỐNG CHI TIẾT.

(Link figma: https://www.figma.com/make/KPHclQbVVEOxpyHsMfWaDQ/IoT-Dashboard-Web-Interface-7?fullscreen=1&t=n7UGsoYbNuBbPRrO-1&code-node-id=0-6)

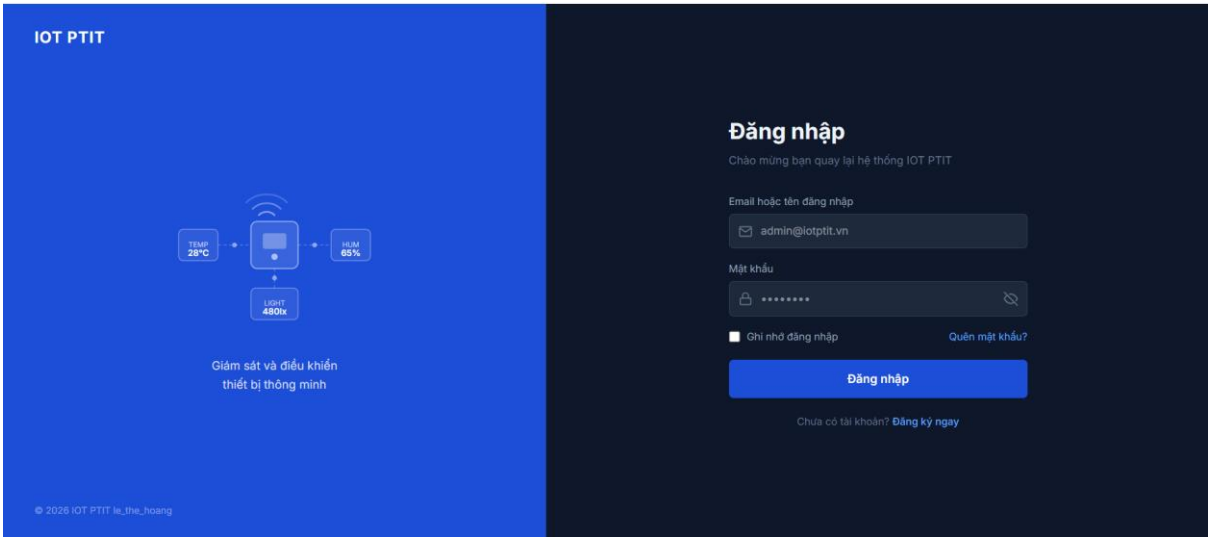
1. Mô tả Usecase và Sequence diagram



Hình 2. Sơ đồ usecase tổng quát

1.1. UC01: Đăng nhập

a) Giao diện

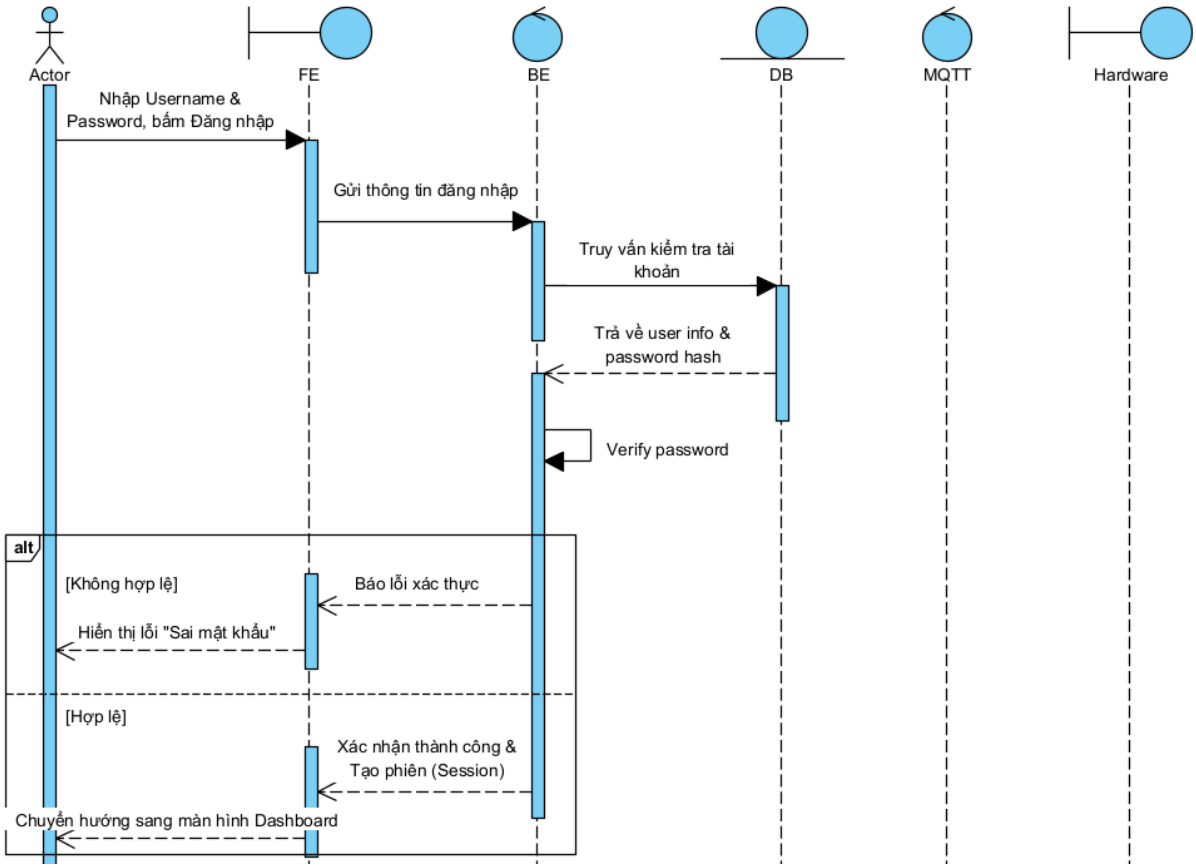


Hình 3. Giao diện đăng nhập.

b) Mô tả Usecase 01: Đăng nhập

Mã Use Case	UC-01
Tên Use Case	Đăng nhập
Tác nhân (Actor)	Người dùng (User)
Mô tả	Cho phép người dùng đăng nhập vào hệ thống bằng tài khoản đã được cấp để truy cập các chức năng giám sát và điều khiển thiết bị.
Điều kiện tiên quyết	Người dùng đã có tài khoản (username/password) hợp lệ trong bảng Users; chưa có phiên đăng nhập (token) hợp lệ.
Luồng sự kiện chính	1. Người dùng truy cập trang đăng nhập, nhập Tên đăng nhập và Mật khẩu. 2. Người dùng nhấn nút "Đăng nhập". 3. Frontend gửi thông tin (Tên đăng nhập, Mật khẩu) lên Backend để yêu cầu đăng nhập. (api dùng: POST /api/auth/login {username, password}) 4. Backend truy vấn bảng Users trong Cơ sở dữ liệu để tìm kiếm tài khoản. 5. Backend đối chiếu mật khẩu người dùng nhập vào với mật khẩu trong Cơ sở dữ liệu. 6. Nếu khớp, Backend báo thành công và tạo một phiên làm việc (Session) cho người dùng. 7. Frontend ghi nhận phiên đăng nhập và cho phép chuyển hướng vào trang Dashboard.
Luồng thay thế / Ngoại lệ	• Sai tên đăng nhập hoặc mật khẩu: Hệ thống từ chối truy cập, hiển thị thông báo lỗi "Sai tên đăng nhập hoặc mật khẩu" và yêu cầu người dùng nhập lại. • Lỗi kết nối máy chủ: Frontend hiển thị thông báo "Không thể kết nối máy chủ", cho phép thử lại.
Hậu điều kiện	Người dùng đăng nhập thành công, phiên làm việc được thiết lập và người dùng có quyền truy cập vào các chức năng giám sát, điều khiển của hệ thống.

c) Sơ đồ tuần tự.



Hình 4. Sơ đồ tuần tự UC01: Đăng nhập

d) Mô tả luồng

Bước 1**:** Người dùng truy cập giao diện đăng nhập, nhập Username, Password và nhấn nút "Đăng nhập".

Bước 2**:** Frontend thu thập thông tin và gửi yêu cầu xác thực POST /api/auth/login tới Backend.

Bước 3**:** Backend tiếp nhận yêu cầu, thực thi câu lệnh SQL truy vấn bảng users (SELECT * FROM users WHERE username = ?).

Bước 4**:** Database xử lý và trả về thông tin người dùng kèm mã băm mật khẩu (password hash).

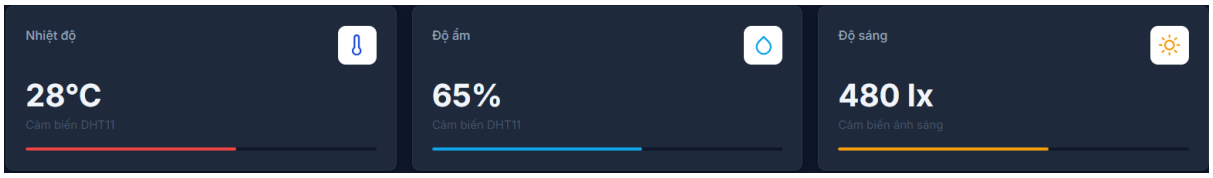
Bước 5**:** Backend tiến hành hàm băm và đối chiếu mật khẩu người dùng gửi lên với mật khẩu mã hóa trong CSDL.

Nhánh điều kiện [alt]:

- [Hợp lệ]:
  - Backend sinh mã phiên/Token JWT, trả thông báo thành công kèm Token về Frontend.
  - Frontend lưu Token vào bộ nhớ trình duyệt (LocalStorage/Cookie) và chuyển hướng người dùng sang trang Dashboard.
- [Không hợp lệ]:
  - Backend trả về mã lỗi HTTP 401 Unauthorized kèm thông báo lỗi.
  - Frontend bắt mã lỗi và hiển thị cảnh báo "Sai tên đăng nhập hoặc mật khẩu" cho người dùng nhập lại.

1.2. UC02: Theo dõi số liệu cảm biến đo ở thời gian thực

a) Giao diện



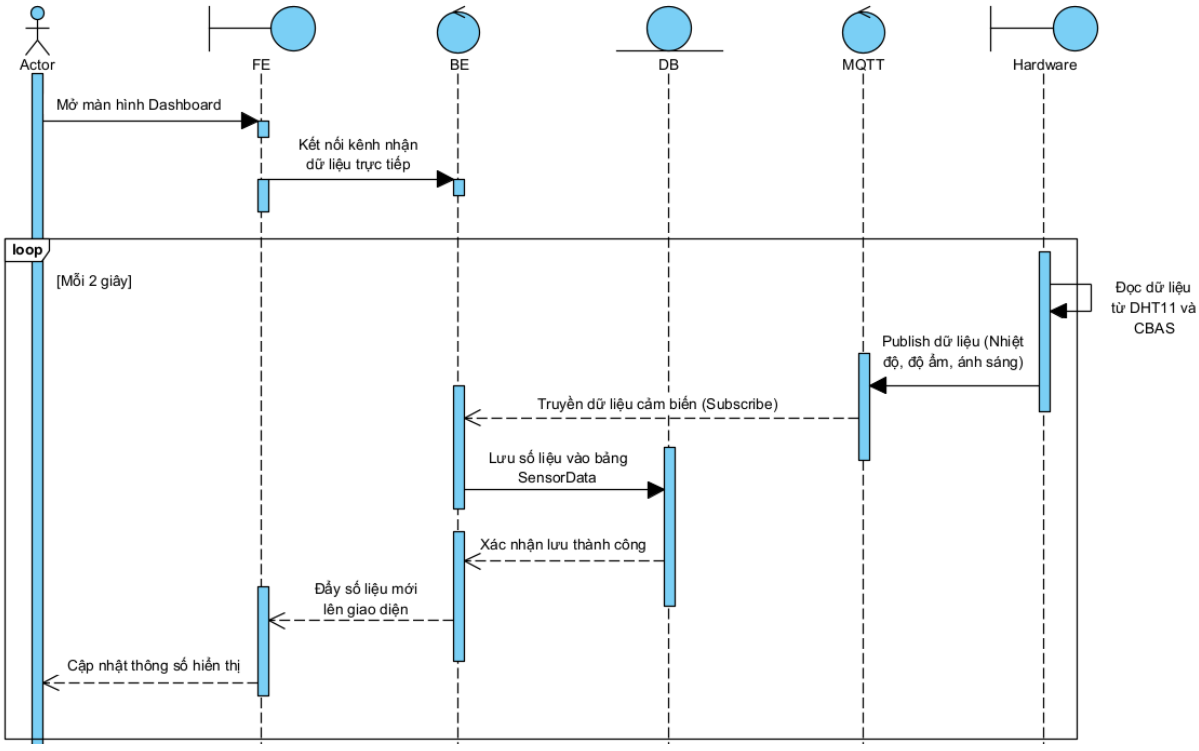
Hình 5. Giao diện theo dõi số liệu cảm biến thời gian thực

b) Mô tả Usecase 02: Theo dõi số liệu cảm biến đo ở thời gian thực

Mã Use Case	UC-02
Tên Use Case	Xem số liệu CB đo thời gian thực
Tác nhân (Actor)	Người dùng (đã đăng nhập), Thiết bị phần cứng (ESP8266)
Mô tả	Thu thập và hiển thị số liệu nhiệt độ, độ ẩm, ánh sáng theo thời gian thực trên Dashboard, cập nhật định kỳ 2 giây/lần.
Điều kiện tiên quyết	Người dùng đã đăng nhập vào hệ thống; phần cứng (ESP8266) đã kết nối mạng internet và sẵn sàng truyền dữ liệu.
Luồng sự kiện chính	1. Người dùng mở trang Dashboard. 2. Giao diện web thiết lập luồng kết nối liên tục với máy chủ để chờ nhận dữ liệu. (Qua giao thức WebSocket: /ws/realtime) 3. (Lặp lại mỗi 2 giây) ESP8266 đọc giá trị từ cảm biến DHT11 (nhiệt độ, độ ẩm) và LDR (ánh sáng).

	4. Phần cứng gửi gói dữ liệu này lên máy chủ thông qua giao thức MQTT. 5. Backend tiếp nhận dữ liệu và lưu trữ vào Cơ sở dữ liệu. 6. Backend đẩy (emit) dữ liệu mới nhất tới Frontend qua WebSocket. 7. Frontend cập nhật số liệu hiển thị trên giao diện ngay lập tức.
Luồng thay thế / Ngoại lệ	• Mất kết nối MQTT/phần cứng: không có dữ liệu mới trong khoảng thời gian quy định → Frontend hiển thị cảnh báo "Mất kết nối cảm biến". • Mất kết nối WebSocket: Frontend tự động thử kết nối lại (reconnect) theo chu kỳ.
Hậu điều kiện	Dữ liệu cảm biến mới nhất được lưu vào cơ sở dữ liệu (bảng SensorData) và hiển thị liên tục trên giao diện người dùng.

c) Sequence diagram.



Hình 6. Sơ đồ tuần tự UC02: Theo dõi số liệu cảm biến đo ở thời gian thực

d) Mô tả luồng

-Khởi tạo kết nối: Người dùng mở màn hình Dashboard, Frontend khởi tạo và duy trì một kênh kết nối WebSocket liên tục tới Backend (/ws/realtime).

-Vòng lặp thu thập định kỳ (loop - Mỗi 2 giây):

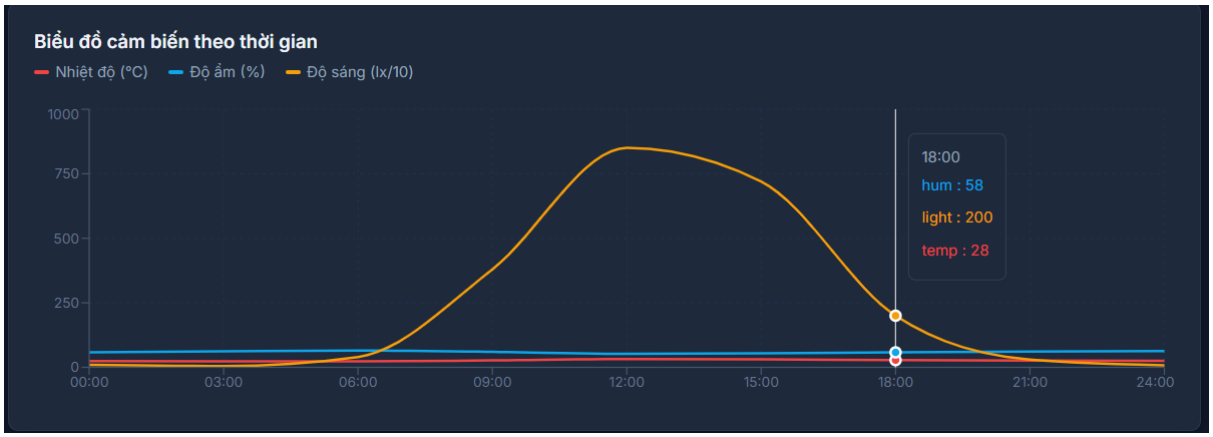
- Bước 1: Vi điều khiển ESP8266 đọc tín hiệu vật lý từ cảm biến DHT11 (nhiệt

độ, độ ẩm) và cảm biến ánh sáng (độ sáng).

- Bước 2: ESP8266 Publish gói dữ liệu môi trường vừa thu thập lên MQTT Broker.
- Bước 3: MQTT Broker chuyển tiếp bản tin tới Backend (do Backend đã subscribe topic tương ứng).
- Bước 4: Backend trích xuất các giá trị và thực hiện câu lệnh chèn dữ liệu (INSERT INTO sensordata) vào CSDL.
- Bước 5: Database phản hồi xác nhận lưu trữ thành công cho Backend.
- Bước 6: Backend lập tức phát sự kiện đẩy gói dữ liệu mới nhất này qua kết nối WebSocket tới Frontend.
- Bước 7: Frontend nhận dữ liệu và render giá trị tức thời lên 3 box thông số (Nhiệt độ, Độ ẩm, Ánh sáng) trên giao diện.

1.3. UC03: Hiển thị biểu đồ dữ liệu cảm biến

a) Giao diện



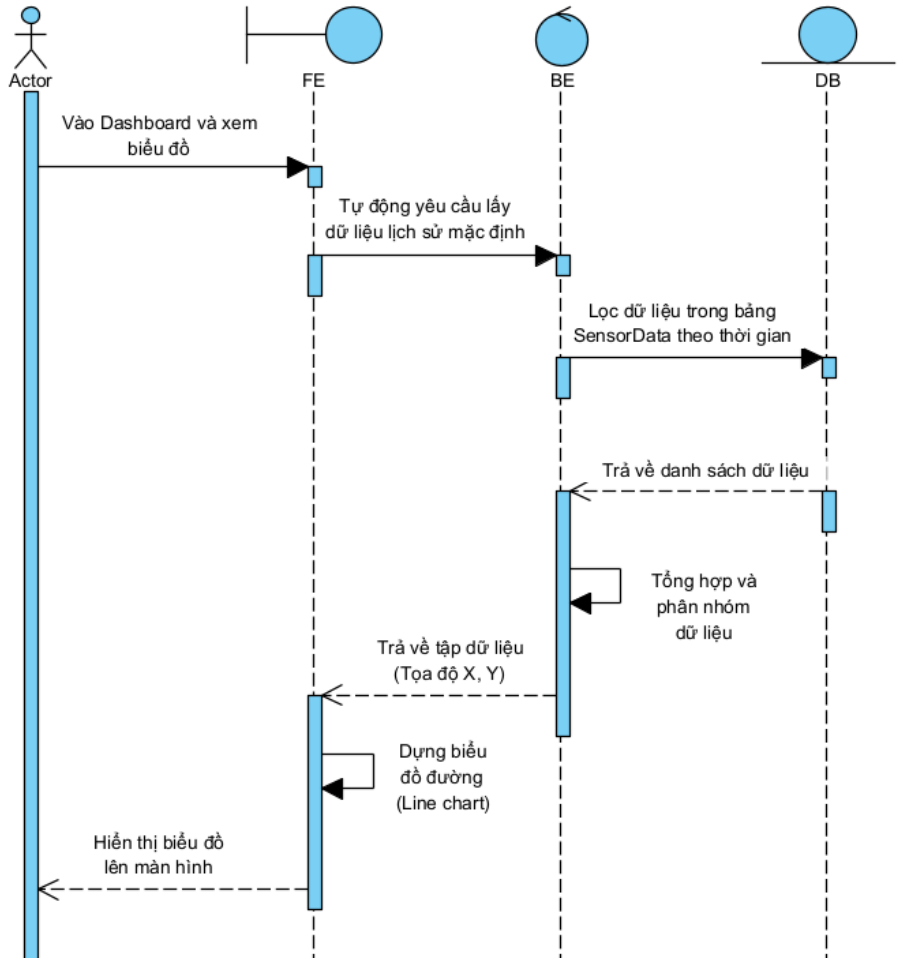
Hình 7. Giao diện biểu đồ dữ liệu cảm biến

b) Mô tả Usecase 03: Hiển thị biểu đồ dữ liệu cảm biến

Mã Use Case	UC-03
Tên Use Case	Hiển thị biểu đồ dữ liệu cảm biến
Tác nhân (Actor)	Người dùng
Mô tả	Cho phép người dùng xem biểu đồ xu hướng số liệu cảm biến theo thời gian nhằm theo dõi, phân tích diễn biến môi trường.
Điều kiện tiên quyết	Người dùng đã đăng nhập; đã tồn tại dữ liệu lịch sử trong bảng SensorData.
Luồng sự kiện chính	1. Người dùng vào trang Dashboard để xem biểu đồ.

	2. Frontend tự động yêu cầu trích xuất dữ liệu lịch sử mặc định (ví dụ: dữ liệu trong 1 giờ qua) từ máy chủ. (GET /api/sensor-data/chart) 3. Backend tiến hành tìm kiếm và lọc dữ liệu trong Cơ sở dữ liệu theo thời gian mặc định. 4. Cơ sở dữ liệu trả về tập dữ liệu tương ứng. 5. Backend tổng hợp, định dạng dữ liệu JSON (nhãn thời gian + giá trị) trả về Frontend. 6. Frontend vẽ biểu đồ đường (Line Chart) hiển thị lên Dashboard.
Luồng thay thế / Ngoại lệ	• Nếu không có dữ liệu trong khoảng thời gian nhất định: Frontend hiển thị thông báo "Không có dữ liệu để hiển thị".
Hậu điều kiện	Biểu đồ xu hướng được hiển thị chính xác theo bộ lọc mà người dùng đã chọn.

c) Sequence diagram.



Hình 8. Sơ đồ tuần tự UC03: biểu đồ dữ liệu cảm biến

d) Mô tả luồng

Bước 1: Người dùng điều hướng vào màn hình Dashboard và xem phân vùng "Biểu đồ cảm biến theo thời gian".

Bước 2: Frontend tự động gửi yêu cầu GET /api/sensor-data/chart lên Backend để trích xuất tập dữ liệu chuỗi thời gian.

Bước 3: Backend thực hiện truy vấn và tổng hợp dữ liệu (Aggregation) theo khoảng thời gian (ví dụ: gom nhóm theo 15 phút hoặc 1 giờ) để giảm tải số điểm dữ liệu trước khi trả về.

Bước 4: CSDL xử lý và trả về danh sách các bản ghi lịch sử.

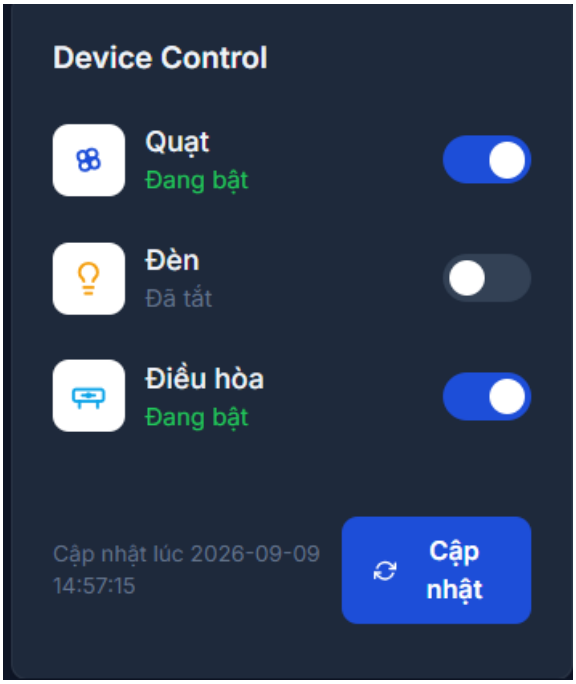
Bước 5: Backend thực hiện hàm tổng hợp, chuẩn hóa các điểm dữ liệu theo mốc thời gian thành định dạng JSON chuẩn (gồm trục hoành thời gian và trục tung giá trị của từng loại cảm biến).

Nhánh điều kiện [alt]:

- [Có dữ liệu]:
    - Backend trả về mảng dữ liệu JSON cho Frontend.
    - Frontend nạp dữ liệu vào thư viện biểu đồ và vẽ biểu đồ đường (Line Chart) đa trục cho người dùng theo dõi trực quan xu hướng.
- [Không có dữ liệu]:
    - Backend trả về mảng rỗng [].
    - Frontend hiển thị trạng thái biểu đồ trống kèm thông báo "Không có dữ liệu trong khoảng thời gian này".

1.4. UC04: Điều khiển bật/tắt thiết bị

a) Giao diện



Hình 9. Giao diện điều khiển bật/tắt thiết bị

b) Mô tả Usecase 04: Điều khiển bật/tắt thiết bị

Mã Use Case	UC-04
Tên Use Case	Điều khiển bật/tắt thiết bị
Tác nhân (Actor)	Người dùng, Thiết bị phần cứng (ESP8266)
Mô tả	Cho phép người dùng kiểm tra trạng thái thực tế mới nhất của các thiết bị (đồng bộ tự động khi tải trang hoặc khi bấm nút "Cập nhật") và thực hiện gửi lệnh Bật/Tắt từ xa (3 đèn LED tượng trưng cho quạt, điều hòa, đèn điện) thông qua giao thức MQTT, có cơ chế xác nhận trạng thái phản hồi vật lý từ phần cứng.
Điều kiện tiên quyết	Người dùng đã đăng nhập hệ thống thành công; vi điều khiển ESP8266 đã kết nối nguồn điện, Wi-Fi và MQTT Broker.
Luồng sự kiện chính	1. Người dùng truy cập Dashboard hoặc nhấn nút "Cập nhật" tại khối Device Control. 2. Frontend gửi yêu cầu GET /api/devices tới Backend để lấy trạng thái mới nhất. 3. Backend truy vấn bảng Devices trong Database và trả về danh sách trạng thái hiện hành. 4. Frontend cập nhật lại các nút gạt (Toggle), nhãn trạng thái ("Đang

	bật"/"Đã tắt") và hiển thị mốc thời gian làm mới. 5. Người dùng quan sát thấy trạng thái đã đồng bộ và gạt nút Bật/Tắt trên thẻ thiết bị tương ứng. 6. Frontend gửi lệnh điều khiển POST /api/devices/{id}/control (kèm trạng thái mong muốn: ON/OFF) tới Backend. 7. Backend ghi nhận một bản ghi mới vào bảng Actions với trạng thái ban đầu là PENDING. 8. Backend phát hành lệnh điều khiển lên MQTT Broker theo topic iot/device/control/{id}. 9. MQTT Broker chuyển tiếp lệnh điều khiển tới mạch ESP8266. 10. ESP8266 xuất tín hiệu logic ra chân GPIO điều khiển đèn LED và publish trạng thái phản hồi thực tế lên topic iot/device/state/{id}. 11. MQTT Broker tiếp nhận bản tin phản hồi và chuyển tiếp về Backend. 12. Backend cập nhật actual_state trong bảng Devices và chuyển trạng thái bản ghi trong bảng Actions từ PENDING thành SUCCESS. 13. Backend phản hồi kết quả thành công về cho Frontend. 14. Frontend cập nhật vị trí nút gạt, đổi nhãn trạng thái, cập nhật dòng thời gian làm mới và hiển thị thông báo "Điều khiển thiết bị thành công" cho Người dùng.
Luồng thay thế / Ngoại lệ	• Mất kết nối phần cứng / Timeout: Quá thời gian quy định mà Backend không nhận được bản tin phản hồi từ ESP8266, Backend tự động đánh dấu bản ghi trong bảng Actions thành FAILED với nguyên nhân lỗi Timeout. Backend trả về mã lỗi cho Frontend; Frontend tự động gạt nút bấm về trạng thái ban đầu và hiển thị thông báo cảnh báo lỗi "Thiết bị không phản hồi, vui lòng kiểm tra phần cứng" cho Người dùng. • Mất kết nối máy chủ: Frontend thông báo "Không thể kết nối máy chủ", giữ nguyên trạng thái hiển thị hiện hành.
Hậu điều kiện	Trạng thái vật lý của thiết bị được thay đổi chính xác theo mong muốn; bảng Devices lưu trạng thái thực tế mới nhất; bảng Actions ghi nhận đầy đủ lịch sử lệnh cùng kết quả thực thi.

c) Sequence diagram



Hình 10. Sơ đồ tuần tự UC04: Điều khiển bật/tắt thiết bị

d) Mô tả luồng

Bước 1: Người dùng mở màn hình Dashboard hoặc chủ động nhấn nút "Cập nhật" tại khối Điều khiển thiết bị (Device Control).

Bước 2: Frontend tiếp nhận thao tác và gửi yêu cầu gọi API GET /api/devices tới

Backend để lấy danh sách trạng thái hiện tại của toàn bộ thiết bị.

Bước 3: Backend thực hiện câu lệnh truy vấn cơ sở dữ liệu (SELECT device_id, device_name, actual_state FROM Devices) xuống Database.

Bước 4: Database xử lý truy vấn và trả về danh sách bản ghi trạng thái thiết bị cho Backend.

Bước 5: Backend đóng gói dữ liệu và trả về phản hồi dạng JSON cho Frontend.

Bước 6: Frontend render vị trí các nút gạt tương ứng (ON/OFF), nhãn trạng thái trực quan và cập nhật dòng chữ hiển thị mốc thời gian vừa làm mới (Ví dụ: Cập nhật lúc 2026-09-08 22:12:06) cho Người dùng quan sát.

Bước 7: Người dùng nhấn nút gạt (Toggle Switch) của thiết bị muốn điều khiển trên giao diện Web.

Bước 8: Frontend gửi yêu cầu điều khiển POST /api/devices/{id}/control (kèm dữ liệu command: ON hoặc command: OFF) tới Backend.

Bước 9: Backend tiếp nhận yêu cầu và thực hiện thêm mới một bản ghi vào bảng Actions trên Database với trạng thái đang chờ xử lý (status: PENDING).

Bước 10: Database ghi nhận bản ghi và phản hồi xác nhận lưu thành công về cho Backend.

Bước 11: Backend đóng gói payload và phát hành (Publish) bản tin điều khiển lên MQTT Broker theo chủ đề iot/device/control/{id}.

Bước 12: MQTT Broker chuyển tiếp bản tin điều khiển tới mạch vi điều khiển ESP8266 (đã đăng ký Subscribe topic này).

Bước 13: Vi điều khiển ESP8266 thực thi xuất mức điện áp logic (HIGH/LOW) ra chân GPIO tương ứng để Bật/Tắt đèn LED thực tế.

Bước 14: Sau khi tác động vật lý, ESP8266 Publish bản tin phản hồi mang trạng thái thực tế lên MQTT Broker theo topic iot/device/state/{id}.

Bước 15: MQTT Broker tiếp nhận bản tin phản hồi và forward trực tiếp về cho Backend.

Bước 16: Hệ thống phân nhánh xử lý kết quả trong khối điều kiện alt:

- Trường hợp Thực thi thành công:
    - Backend cập nhật trạng thái mới cho thiết bị trong bảng Devices (actual_state), đồng thời cập nhật bản ghi trong bảng Actions thành thành công (status: SUCCESS) xuống Database.

- Database xác nhận cập nhật thành công.
- Backend gửi phản hồi HTTP 200 OK kèm dữ liệu thành công về cho Frontend.
- Frontend chuyển đổi vị trí nút gạt, cập nhật nhãn trạng thái ("Đang bật" / "Đã tắt"), cập nhật lại mốc thời gian làm mới và hiển thị thông báo "Điều khiển thiết bị thành công" cho Người dùng.

- Trường hợp Mất kết nối / Quá thời gian (Timeout):
  - Nếu quá thời gian quy định Backend không nhận được tín hiệu phản hồi từ phần cứng, Backend tiến hành cập nhật bảng Actions với trạng thái thất bại (status: FAILED, nguyên nhân: Timeout) xuống Database và giữ nguyên trạng thái cũ trong bảng Devices.
  - Database ghi nhận lỗi và phản hồi lại.
  - Backend trả về thông báo lỗi Timeout cho Frontend.
  - Frontend hoàn tác trạng thái nút gạt về vị trí ban đầu và hiển thị thông báo cảnh báo lỗi "Thiết bị không phản hồi, vui lòng kiểm tra kết nối phần cứng" trực tiếp cho Người dung.

1.5. UC05: Xem lịch sử cảm biến đo số liệu

a) Giao diện

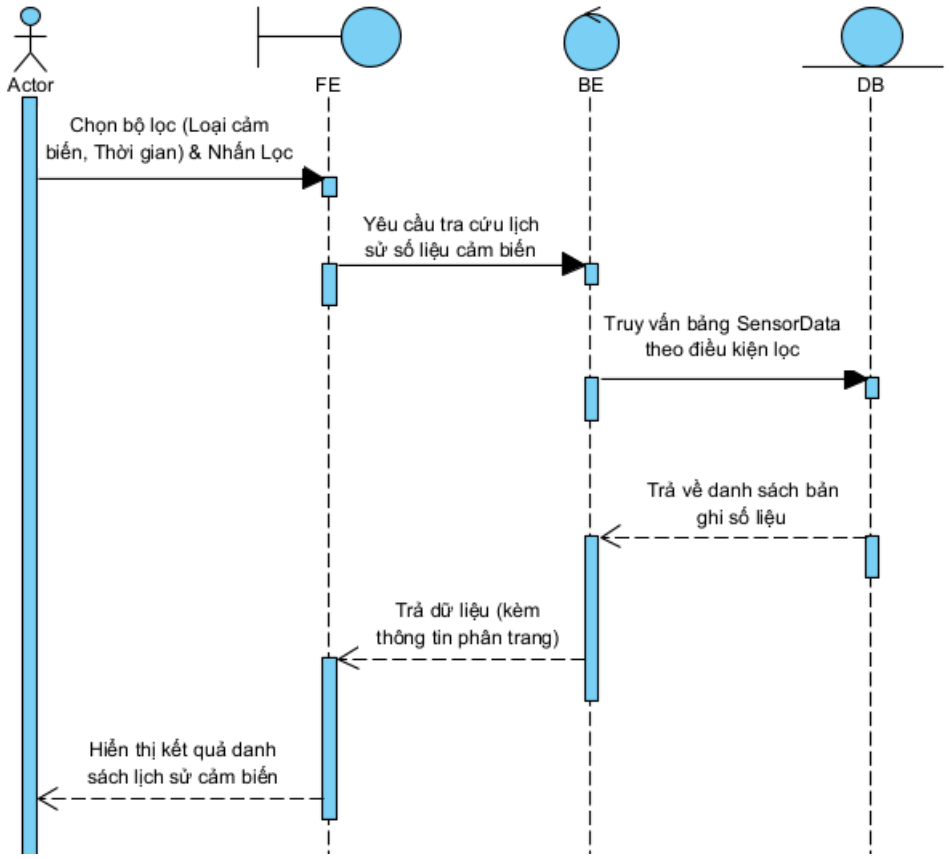


Hình 11. Giao diện Xem lịch sử cảm biến đo số liệu

b) Mô tả Usecase 05: Xem lịch sử cảm biến đo số liệu

Mã Use Case	UC-05
Tên Use Case	Xem lịch sử cảm biến đo số liệu
Tác nhân (Actor)	Người dùng
Mô tả	Cho phép người dùng tra cứu, lọc và xem lại lịch sử số liệu cảm biến đã được ghi nhận trong hệ thống.
Điều kiện tiên quyết	Người dùng đã đăng nhập.
Luồng sự kiện chính	1. Người dùng vào trang Lịch sử cảm biến, chọn bộ lọc (loại cảm biến, khoảng thời gian, số trang) để tìm kiếm. 2. Frontend gửi yêu cầu tra cứu lịch sử dữ liệu tới Backend. (GET /api/sensor-data/history). 3. Backend truy vấn bảng SensorData (JOIN Sensors) theo điều kiện lọc, có phân trang (LIMIT/OFFSET). 4. Cơ sở dữ liệu trả về danh sách bản ghi cùng tổng số dòng phù hợp. 5. Backend đóng gói danh sách dữ liệu và thông tin số trang, sau đó gửi trả về cho Frontend. 6. Frontend hiển thị bảng dữ liệu lịch sử kèm điều hướng phân trang.
Luồng thay thế / Ngoại lệ	• Nếu không có dữ liệu phù hợp với bộ lọc: Frontend hiển thị bảng trống kèm thông báo "Không tìm thấy dữ liệu".
Hậu điều kiện	Danh sách dữ liệu lịch sử được hiển thị chính xác theo bộ lọc người dùng đã chọn.

c) Sequence diagram.



Hình 12. Sơ đồ tuần tự UC05: Xem lịch sử cảm biến đo số liệu

d) Mô tả luồng

Bước 1: Người dùng truy cập trang "Sensor Data" từ menu điều hướng, chọn các tiêu chí lọc (loại cảm biến, khoảng thời gian, từ khóa tìm kiếm) hoặc chọn số trang cần xem.

Bước 2: Frontend tổng hợp các tham số và gửi yêu cầu GET /api/sensor-data/history?page=...&limit=... lên Backend.

Bước 3: Backend xử lý tham số, thực thi câu lệnh truy vấn có phân trang kết hợp bảng: SELECT sensordata.*, sensors.sensor_name, sensors.unit FROM sensordata JOIN sensors ON sensordata.sensor_id = sensors.sensor_id WHERE ... ORDER BY recorded_at DESC LIMIT ... OFFSET .... Đồng thời đếm tổng số dòng thỏa mãn (COUNT(*)).

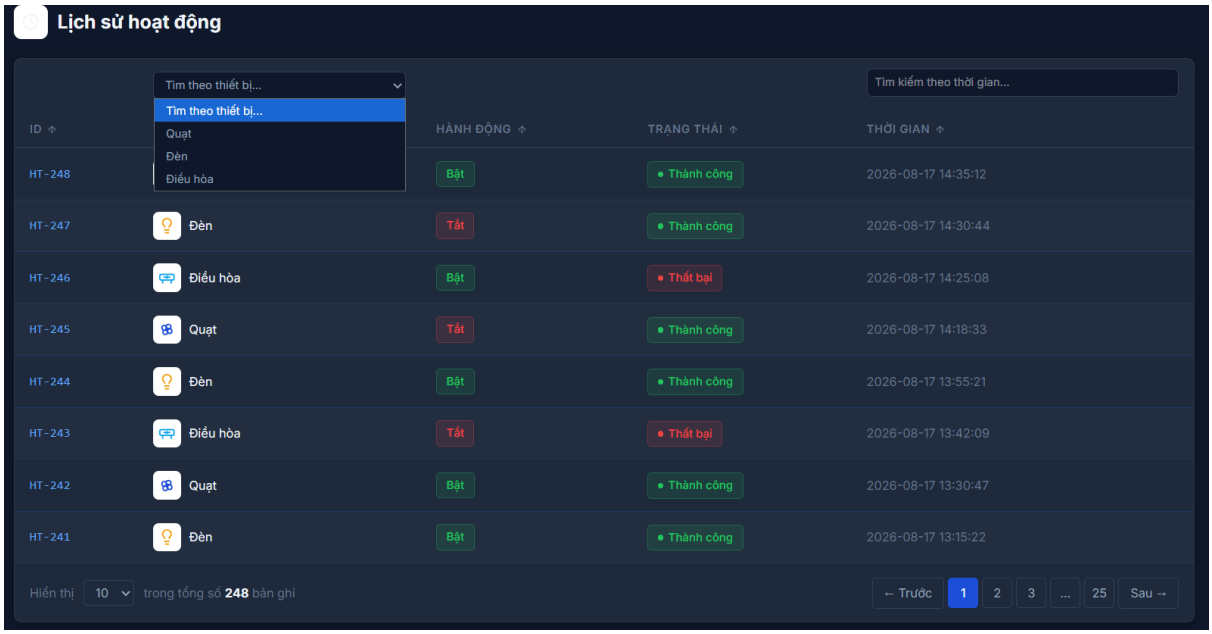
Bước 4: Database trả về danh sách bản ghi và tổng số lượng bản ghi cho Backend.

Nhánh điều kiện [alt]:

- [Có dữ liệu]:
    - Backend đóng gói danh sách bản ghi kèm thông tin phân trang (total, page, totalPages) dưới dạng JSON trả về cho Frontend.
    - Frontend render dữ liệu ra bảng hiển thị gồm các cột: ID, Loại cảm biến, Giá trị đo, Thời gian ghi nhận, cùng thanh phân trang.
- [Không tìm thấy dữ liệu]:
    - Backend trả về mảng rỗng [] kèm total = 0.
    - Frontend hiển thị bảng trống kèm thông báo "Không tìm thấy dữ liệu phù hợp với bộ lọc".

1.6. UC06: Xem lịch sử bật/tắt thiết bị

a) Giao diện



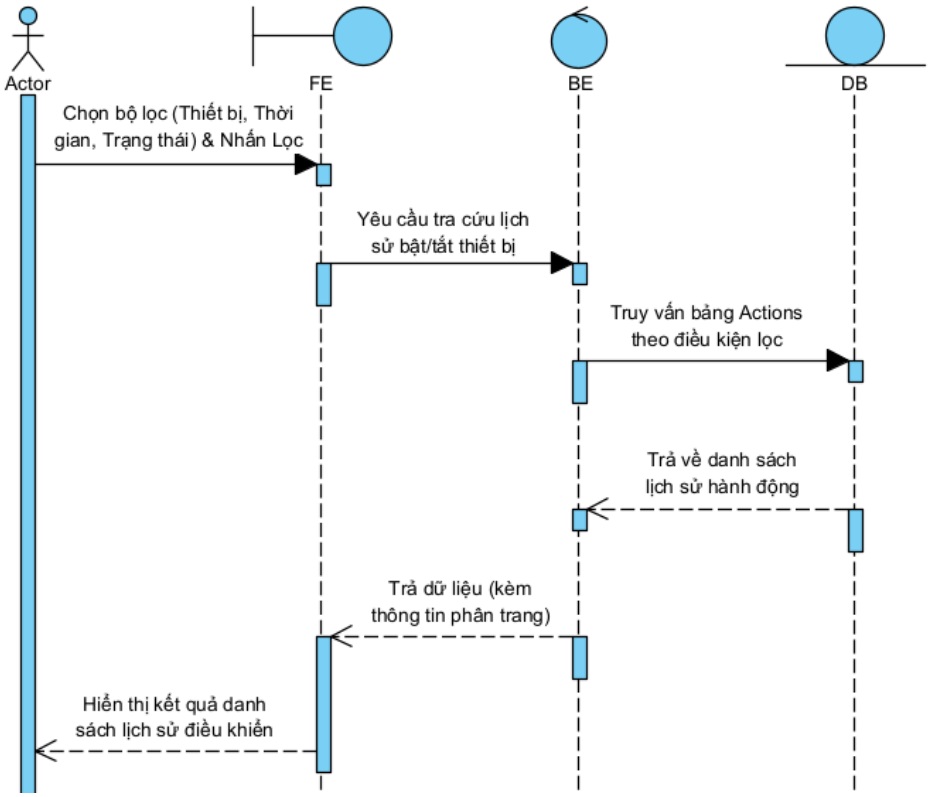
Hình 13.Giao diện Xem lịch sử bật/tắt thiết bị

b) Mô tả Usecase 06: Xem lịch sử bật/tắt thiết bị

Mã Use Case	UC-06
Tên Use Case	Xem lịch sử bật/tắt thiết bị
Tác nhân (Actor)	Người dùng
Mô tả	Cho phép người dùng tra cứu lại lịch sử các lệnh bật/tắt thiết bị đã thực hiện, kèm trạng thái xác nhận thực tế từ phần cứng và người thực hiện.

Điều kiện tiên quyết	Người dùng đã đăng nhập.
Luồng sự kiện chính	1. Người dùng vào trang Lịch sử thiết bị, chọn bộ lọc (thiết bị, khoảng thời gian, trạng thái) rồi tìm kiếm. 2. Frontend gửi yêu cầu tra cứu lịch sử dữ liệu tới Backend. (GET /api/actions/history). 3. Backend truy vấn bảng Actions (JOIN Devices, Users) theo điều kiện lọc, có phân trang. 4. Cơ sở dữ liệu trả về danh sách hành động (lệnh, trạng thái, người thực hiện, thời gian). 5. Backend đóng gói danh sách dữ liệu và thông tin số trang, sau đó gửi trả về cho Frontend. 6. Frontend hiển thị bảng lịch sử bật/tắt kèm trạng thái xác nhận (Thành công / Đang chờ / Thất bại).
Luồng thay thế / Ngoại lệ	• Không có dữ liệu phù hợp với bộ lọc: Frontend hiển thị bảng trống kèm thông báo "Không tìm thấy dữ liệu".
Hậu điều kiện	Danh sách lịch sử hành động điều khiển thiết bị được hiển thị đầy đủ và chính xác trạng thái.

c) Sequence diagram



Hình 14.Sơ đồ tuần tự UC06: Xem lịch sử bật/tắt thiết bị

d) Mô tả luồng

Bước 1: Người dùng chọn mục "History" trên thanh điều hướng, thiết lập các bộ lọc tra cứu (ID thiết bị, loại lệnh Bật/Tắt, trạng thái Thành công/Thất bại/Đang xử lý, mốc thời gian).

Bước 2: Frontend gửi yêu cầu GET /api/actions/history?page=...&limit=... lên Backend.

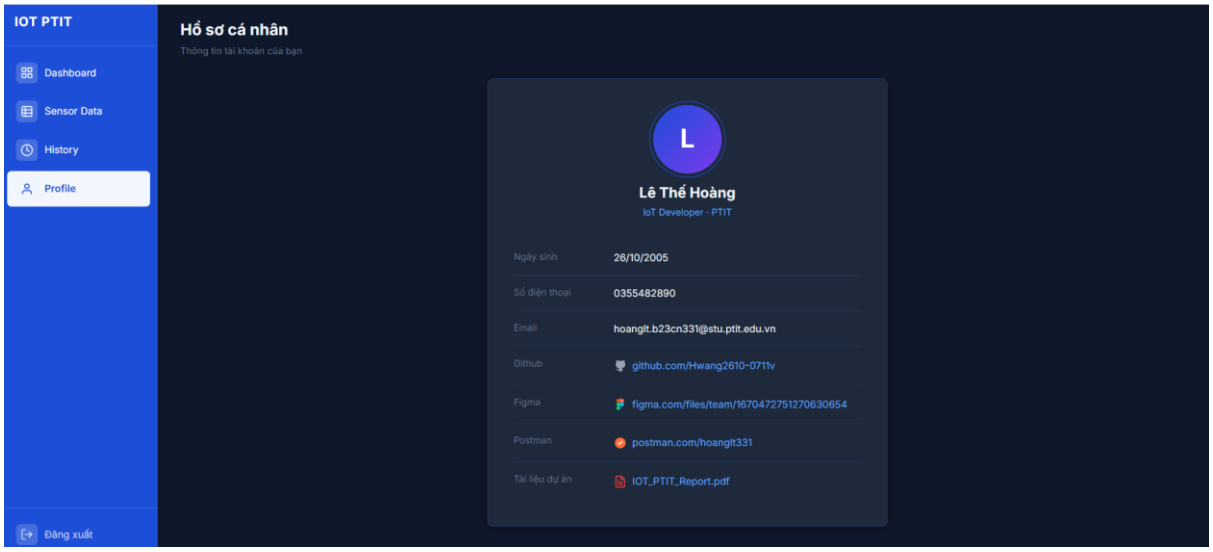
Bước 3: Backend tạo và thực thi câu lệnh truy vấn: SELECT actions.*, devices.device_name, users.username FROM actions JOIN devices ON actions.device_id = devices.device_id JOIN users ON actions.user_id = users.user_id WHERE ... ORDER BY executed_at DESC LIMIT ... OFFSET ....

Bước 4: Database trả về tập kết quả thỏa mãn điều kiện lọc và tổng số dòng cho Backend.

Nhánh điều kiện [alt]:

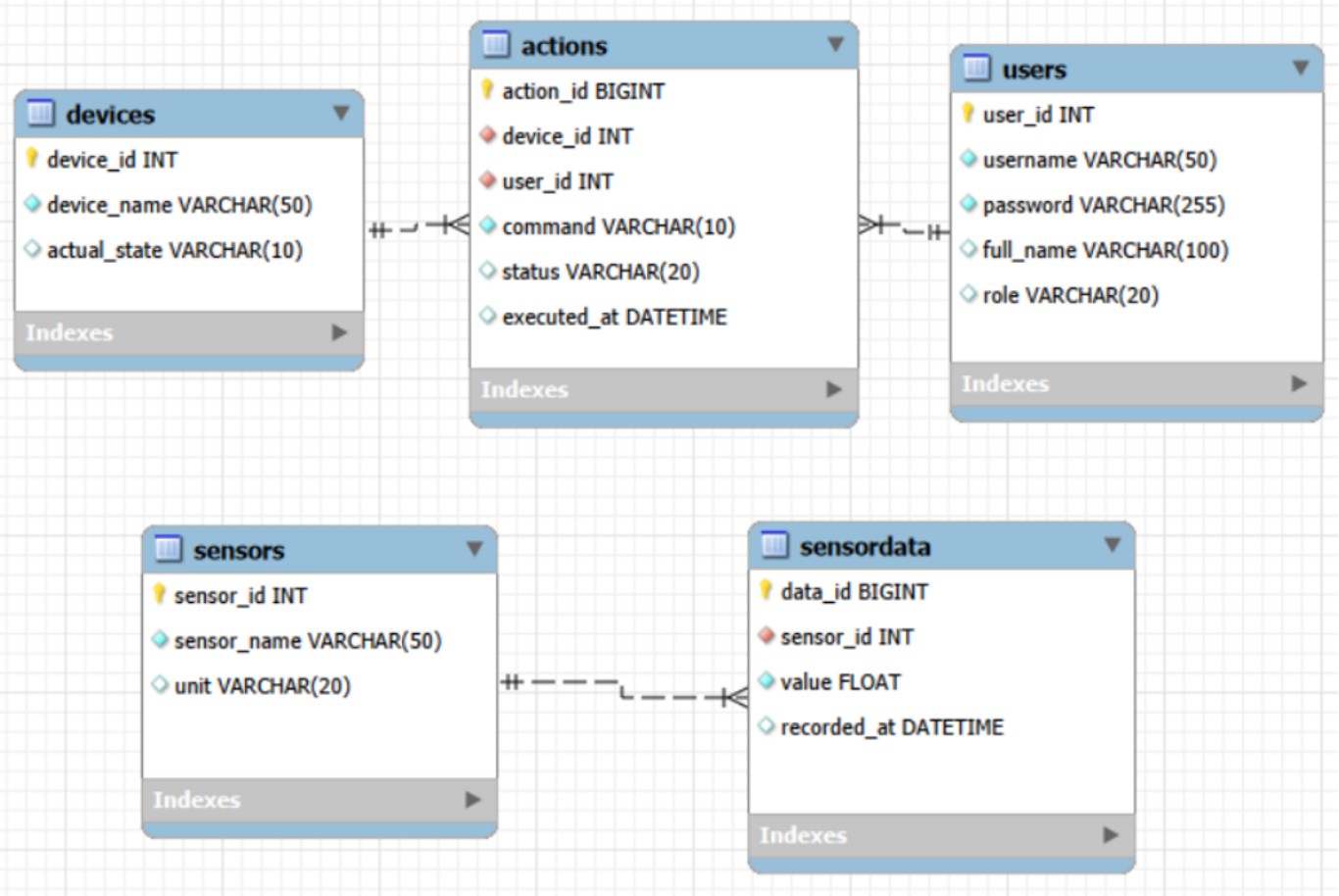
- [Có dữ liệu]:
    - Backend đóng gói mảng danh sách hành động và dữ liệu metadata phân trang gửi về Frontend.
    - Frontend render danh sách ra bảng lịch sử hoạt động gồm các cột: ID, Tên thiết bị, Hành động, Trạng thái (Thành công gắn thẻ xanh, Thất bại thẻ đỏ, Đang xử lý thẻ cam), Thời gian kích hoạt.
- [Không có dữ liệu]:
    - Backend trả về mảng rỗng [].
    - Frontend hiển thị bảng trống cùng thông báo "Không tìm thấy lịch sử thao tác phù hợp".

1.7. Giao diện trang Profile



Hình 15.Giao diện trang profile

2. Database và sơ đồ ERD



Hình 16. Sơ đồ ERD

2.1. Bảng Users

Lưu thông tin tài khoản người dùng, phục vụ chức năng đăng nhập.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
user_id	INT	Primary Key, Auto Increment	Mã định danh người dùng
username	VARCHAR(50)	Not Null, Unique	Tên đăng nhập
password	VARCHAR(255)	Not Null	Mật khẩu đã mã hóa (hash)
full_name	VARCHAR(100)	Nullable	Họ và tên người dùng
role	VARCHAR(20)	Default: 'user'	Phân quyền người dùng (user/admin)

2.2. Bảng Sensors

Lưu danh mục các loại cảm biến có trong hệ thống.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả

sensor_id	INT	Primary Key, Auto Increment	Mã định danh loại cảm biến
sensor_name	VARCHAR(50)	Not Null, Unique	Tên cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng)
unit	VARCHAR(20)	Nullable	Đơn vị đo (°C, %, lux)

2.3. Bảng SensorData

Lưu lịch sử giá trị đo được của từng cảm biến, phục vụ Dashboard, biểu đồ và tra cứu lịch sử (UC02, UC03, UC05).

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
data_id	BIGINT	Primary Key, Auto Increment	Mã bản ghi số liệu
sensor_id	INT	Not Null, Foreign Key (Sensors.sensor_id)	Cảm biến đo được giá trị
value	FLOAT	Not Null	Giá trị đo đạc
recorded_at	DATETIME	Default: CURRENT_TIMESTAMP	Thời điểm ghi nhận dữ liệu

2.4. Bảng Devices

Lưu danh mục thiết bị và trạng thái thực tế.

Tên trường	Kiểu dữ liệu	Ràng buộc	Mô tả
device_id	INT	Primary Key, Auto Increment	Mã định danh thiết bị
device_name	VARCHAR(50)	Not Null	Tên hiển thị thiết bị (Đèn 1, Quạt, Điều hòa...)
actual_state	VARCHAR(10)	Default: 'OFF'	Trạng thái thực tế hiện tại (ON/OFF)

2.5. Bảng Actions

Lưu lịch sử điều khiển bật/tắt thiết bị, phục vụ màn hình lịch sử điều khiển (UC-04, UC-06).

action_id	BIGINT	Primary Key, Auto Increment	Mã bản ghi lịch sử điều khiển
device_id	INT	Not Null, Foreign Key (Devices.device_id)	Thiết bị nhận lệnh điều khiển
user_id	INT	Not Null, Foreign Key (Users.user_id)	Người thực hiện lệnh điều khiển

command	VARCHAR(10)	Not Null	Lệnh điều khiển (ON/OFF)
status	VARCHAR(20)	Default: 'PENDING'	Trạng thái xử lý (PENDING/SUCCESS/FAILED)
executed_at	DATETIME	Default: CURRENT_TIMESTAMP	Thời điểm ghi nhận lệnh

3. API

3.1. API 1: Đăng nhập hệ thống (Authentication)

-Method & Path: POST /api/auth/login

-Mục đích: Xác thực tài khoản người dùng và cấp mã phiên (JWT Token) để truy cập hệ thống.

-Request Body:

JSON

```
{
  "username": "user001",
  "password": "userpassword123"
}
```

-Định dạng dữ liệu trả về:

- *Thành công (HTTP 200 OK):*

JSON

```
{
  "success": true,
  "message": "Đăng nhập thành công",
  "data": {
    "user_id": 1,
    "username": "user001",
    "full_name": "Lê Thế Hoàng",
    "role": "user",
    "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
  }
```

}

- *Thất bại (HTTP 401 Unauthorized):*

JSON

```
{
  "success": false,
  "message": "Tên đăng nhập hoặc mật khẩu không chính xác"
}
```

3.2. Kênh kết nối WebSocket: Đẩy số liệu thời gian thực (Real-time Stream)

-Giao thức & Đường dẫn: WS /ws/realtime

-Mục đích: Kênh truyền thông song công. Backend tự động đẩy các giá trị đo cảm biến mới nhất từ ESP8266 (chu kỳ 2s/lần) và trạng thái các thiết bị về Frontend mà không cần Client phải gửi request định kỳ.

-Dữ liệu đẩy về Client định kỳ (Server-to-Client Payload):

JSON

```
{
  "event": "SENSOR_UPDATE",
  "timestamp": "2026-08-17T14:35:00",
  "data": {
    "temperature": 28.4,
    "humidity": 65.2,
    "light": 480,
    "devices": [
      { "device_id": 1, "device_name": "Quạt", "actual_state": "ON" },
      { "device_id": 2, "device_name": "Đèn", "actual_state": "OFF" },
      { "device_id": 3, "device_name": "Điều hòa", "actual_state": "ON" }
    ]
  }
}
```

3.3. API 2: Lấy dữ liệu phục vụ biểu đồ giám sát (Monitoring Chart)

-Phương thức & Đường dẫn: GET /api/sensor-data/chart

-Mục đích: Trích xuất chuỗi dữ liệu lịch sử theo mốc thời gian để vẽ biểu đồ đường (Line Chart) trên Dashboard.

-Query Parameters (Tùy chọn):

- duration: Khoảng thời gian tra cứu (mặc định: 1h, tùy chọn: 24h, 7d).

-Định dạng dữ liệu trả về:

Thành công (HTTP 200 OK):

JSON


```
{
  "success": true,
  "data": [
    {
      "time": "14:00:00",
      "temperature": 27.5,
      "humidity": 64.0,
      "light": 450
    },
    {
      "time": "14:15:00",
      "temperature": 28.0,
      "humidity": 65.0,
      "light": 470
    },
    {
      "time": "14:30:00",
      "temperature": 28.4,
      "humidity": 65.2,
      "light": 480
    }
  ]
```

}

3.4. API 3: Truy vấn và phân trang lịch sử dữ liệu cảm biến (Sensor Data)

-Phương thức & Đường dẫn: GET /api/sensor-data/history

-Mục đích: Tra cứu, lọc theo tiêu chí và phân trang danh sách dữ liệu cảm biến tại màn hình Sensor Data.

-Query Parameters:

- page (int, default: 1): Trang hiện tại.
- limit (int, default: 10): Số bản ghi trên mỗi trang.
- sensor_name (string, optional): Lọc theo tên cảm biến (Nhiệt độ, Độ ẩm, Độ sáng).
- startDate / endDate (string, optional): Lọc theo khoảng thời gian ghi nhận.

-Định dạng dữ liệu trả về (HTTP 200 OK):

JSON


```
{
  "success": true,
  "data": [
    {
      "data_id": "SN-001",
      "sensor_name": "Nhiệt độ (DHT22)",
      "value": 28.4,
      "unit": "°C",
      "recorded_at": "2026-08-17 14:32:01"
    },
    {
      "data_id": "SN-002",
      "sensor_name": "Độ ẩm (DHT22)",
      "value": 65.2,
      "unit": "%",
      "recorded_at": "2026-08-17 14:30:45"
    },
```

```
    {
      "data_id": "SN-003",
      "sensor_name": "Độ sáng (LDR)",
      "value": 480,
      "unit": "lx",
      "recorded_at": "2026-08-17 14:29:10"
    }
  ],
  "pagination": {
    "current_page": 1,
    "limit": 10,
    "total_records": 248,
    "total_pages": 25
  }
}
```


3.5. API 4: Gửi lệnh điều khiển thiết bị (Device Control)

-Phương thức & Đường dẫn: POST /api/devices/{id}/control

-Mục đích: Gửi lệnh bật hoặc tắt một thiết bị ngoại vi cụ thể (Quạt, Đèn, Điều hòa) thông qua MQTT.

-Path Parameter:

- id (int): ID của thiết bị cần điều khiển (1: Quạt, 2: Đèn, 3: Điều hòa).

-Request Body:

JSON


```
{
  "command": "ON"
}
```


-Luồng xử lý (Workflow):

1. Kiểm tra thiết bị trong bảng devices.
2. Tạo bản ghi mới trong bảng actions với trạng thái PENDING.
3. Gửi lệnh lên MQTT topic iot/device/control/{id}.

4. Đợi gói tin xác nhận từ ESP8266 qua MQTT topic iot/device/state/{id} trong tối đa 5 giây.
5. Cập nhật trạng thái SUCCESS hoặc FAILED vào CSDL và phản hồi kết quả về Client.

-Định dạng dữ liệu trả về:

+Lệnh được tiếp nhận xử lý thành công (HTTP 200 OK):

JSON


```
{
  "success": true,
  "message": "Điều khiển thiết bị thành công",
  "data": {
    "action_id": 248,
    "device_id": 1,
    "device_name": "Quạt",
    "command": "ON",
    "status": "SUCCESS",
    "actual_state": "ON",
    "executed_at": "2026-08-17 14:35:12"
  }
}
```


+Lỗi thiết bị không phản hồi (HTTP 504 Gateway Timeout):

JSON


```
{
  "success": false,
  "message": "Thiết bị không phản hồi, lệnh điều khiển thất bại",
  "data": {
    "action_id": 246,
    "device_id": 3,
    "status": "FAILED"
  }
}
```

3.6. API 5: Truy vấn nhật ký lịch sử điều khiển (Action History)

-Phương thức & Đường dẫn: GET /api/actions/history

-Mục đích: Lấy danh sách lịch sử các thao tác bật/tắt thiết bị của người dùng, hỗ trợ lọc và phân trang tại màn hình History.

-Query Parameters:

- page (int, default: 1): Trang hiện tại.
- limit (int, default: 10): Số bản ghi/trang.
- device_name (string, optional): Lọc theo thiết bị (Quạt, Đèn, Điều hòa).
- command (string, optional): Lọc theo lệnh (ON hoặc OFF).
- status (string, optional): Lọc theo trạng thái (SUCCESS, PENDING, FAILED).

-Định dạng dữ liệu trả về (HTTP 200 OK):

JSON


```
{
  "success": true,
  "data": [
    {
      "action_id": "HT-248",
      "device_name": "Quạt",
      "command": "Bật",
      "status": "Thành công",
      "executed_at": "2026-08-17 14:35:12"
    },
    {
      "action_id": "HT-247",
      "device_name": "Đèn",
      "command": "Tắt",
      "status": "Thành công",
      "executed_at": "2026-08-17 14:30:44"
    },
```

```
    {
      "action_id": "HT-246",
      "device_name": "Điều hòa",
      "command": "Bật",
      "status": "Thất bại",
      "executed_at": "2026-08-17 14:25:08"
    }
  ],
  "pagination": {
    "current_page": 1,
    "limit": 10,
    "total_records": 248,
    "total_pages": 25
  }
}
```


3.7. API 6: Lấy danh sách và trạng thái thiết bị

-Method & Path: GET /api/devices

-Mục đích: Lấy trạng thái thực tế hiện tại của 3 thiết bị để hiển thị lên Dashboard.

-Response (200 OK):


```
{
  "success": true,
  "data": [
    { "device_id": 1, "device_name": "Quạt", "actual_state": "ON" },
    { "device_id": 2, "device_name": "Đèn", "actual_state": "OFF" },
    { "device_id": 3, "device_name": "Điều hòa", "actual_state": "ON" }
  ]
}
```


IV. CHƯƠNG 4: XÂY DỰNG VÀ THỰC NGHIỆM HỆ THỐNG

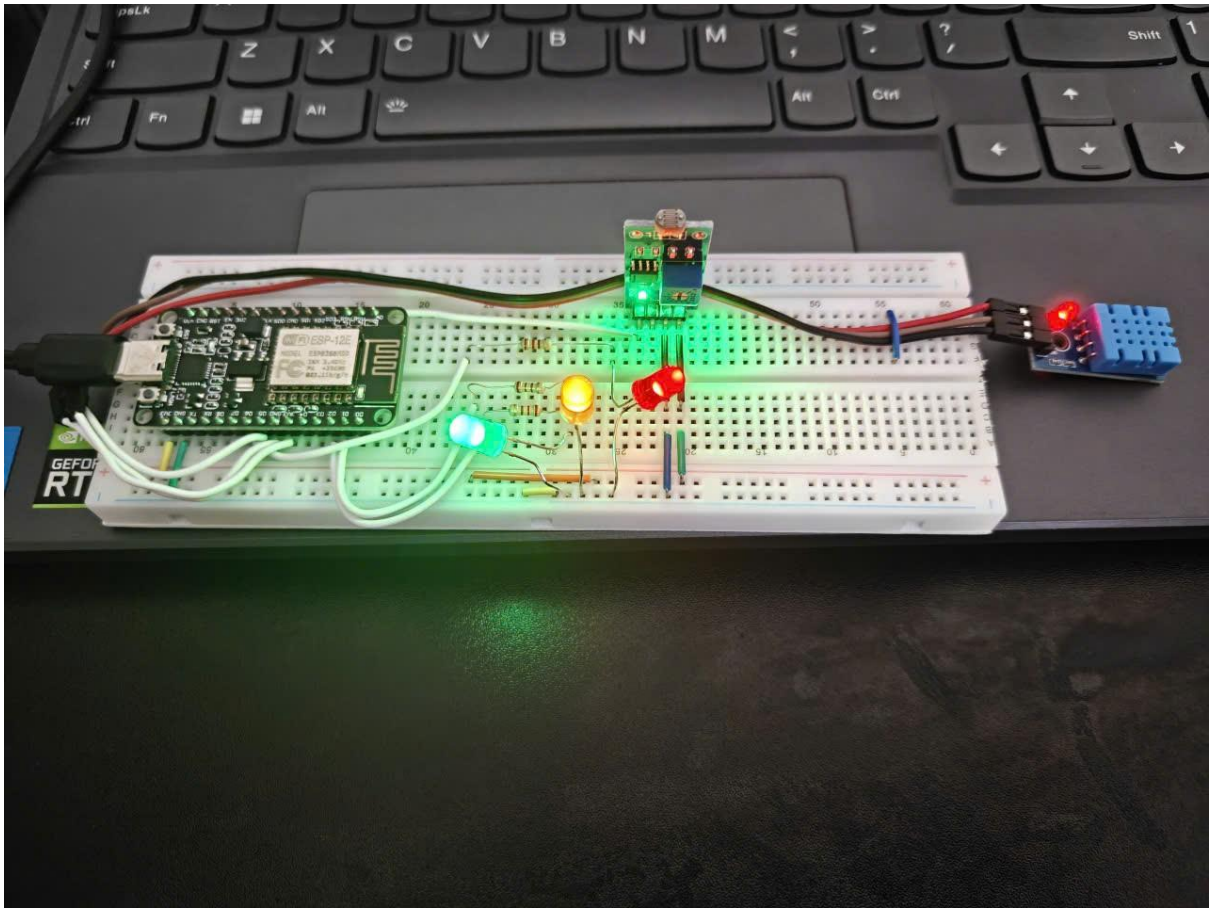
1. Triển khai phần cứng thực tế

1.1. Lắp ráp và kết nối mạch

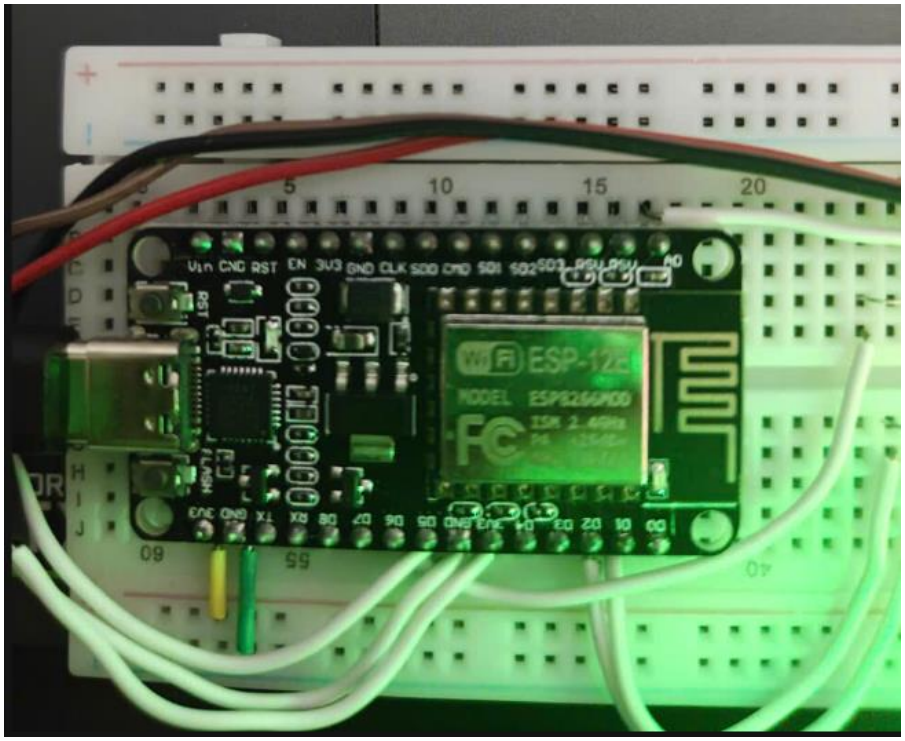
Căn cứ vào sơ đồ nguyên lý và bảng gán chân tín hiệu vi điều khiển đã thiết kế, hệ thống phần cứng thu thập và điều khiển tại biên được lắp ráp thử nghiệm trên bo mạch cắm (Breadboard):

- Khối xử lý trung tâm: Sử dụng bo mạch phát triển ESP8266 NodeMCU (phiên bản chip nạp CH340). Mạch được cấp nguồn hoạt động 5V trực tiếp qua cổng micro-USB kết nối từ máy tính phát triển.
- Khối cảm biến thu thập môi trường:
    - Cảm biến nhiệt độ và độ ẩm DHT11: Chân VCC nối nguồn 3.3V, chân GND nối mass chung, chân dữ liệu (DATA) được nối trực tiếp vào chân số GPIO 14 (D5) của ESP8266.
    - Module cảm biến quang trở LDR: Chân VCC nối nguồn 3.3V, chân GND nối mass chung, chân tín hiệu tương tự (AO - Analog Out) được nối vào chân A0 (ADC0) của ESP8266 nhằm thu thập mức điện áp biến thiên theo cường độ ánh sáng môi trường.
- Khối cơ cấu chấp hành (3 kênh đầu ra mô phỏng):
    - Đèn LED 1 (Mô phỏng Quạt): Cực dương (Anode) nối vào chân GPIO 5 (D1), cực âm (Cathode) nối tiếp qua điện trở hạn dòng 220Ω về mass (GND).
    - Đèn LED 2 (Mô phỏng Đèn chiếu sáng): Cực dương nối vào chân GPIO 4 (D2), cực âm nối tiếp qua trở 220Ω về GND.
    - Đèn LED 3 (Mô phỏng Điều hòa nhiệt độ): Cực dương nối vào chân GPIO 12 (D6), cực âm nối tiếp qua trở 220Ω về GND.

1.2. Hình ảnh mạch chạy thực tế



Hình 4.1. Mô hình phần cứng hệ thống đo môi trường và điều khiển thiết bị trên Breadboard



Hình 4.2. Chi tiết kết nối chân vi điều khiển ESP8266 và khối cảm biến đầu vào

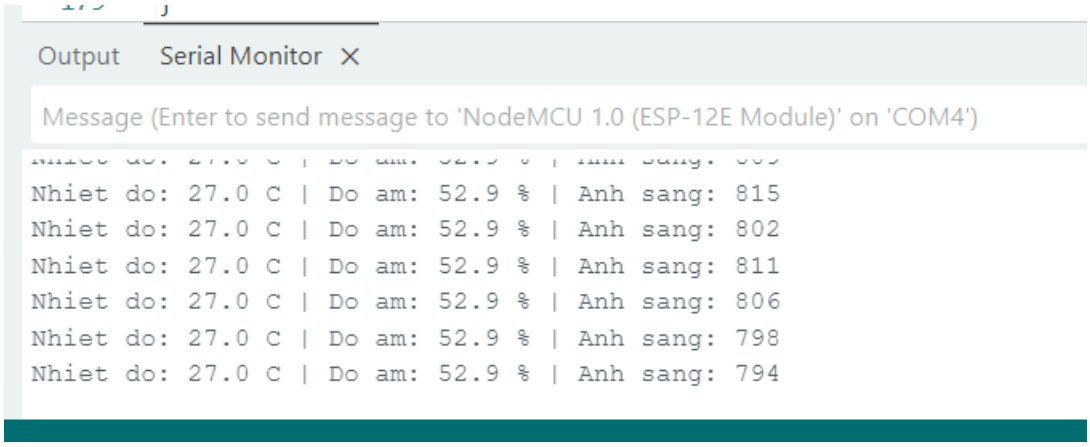
1.3. Kết quả ghi nhận tại Serial Monitor và Terminal

Hệ thống mạng truyền thông sử dụng phần mềm Mosquitto MQTT Broker chạy cục bộ tại địa chỉ IP 10.223.239.92, cổng dịch vụ 1891, yêu cầu xác thực bằng tài khoản sinh viên lethehoang và mật khẩu b23dccn331.

-Kiểm tra luồng gửi dữ liệu cảm biến định kỳ (Topic sensor_data): Vi điều khiển ESP8266 đọc dữ liệu mỗi 2 giây, đóng gói chuỗi thông số môi trường và gửi lên Broker. Sử dụng tiện ích dòng lệnh kiểm tra:

mosquitto_sub -h 10.223.239.92 -p 1891 -t "sensor_data" -v -u "lethehoang" -P "b23dccn331"
-Kết quả bản tin nhận được liên tục theo thời gian thực:

sensor_data Nhiet do: 28.4 C | Do am: 65.2 % | Anh sang: 480
sensor_data Nhiet do: 28.5 C | Do am: 65.0 % | Anh sang: 482
sensor_data Nhiet do: 28.4 C | Do am: 65.2 % | Anh sang: 479



Hình 4.3. Kết quả đo lường cảm biến hiển thị tại Serial Monitor của ESP8266

- Kiểm tra luồng tiếp nhận lệnh điều khiển và phản hồi trạng thái: Khi máy chủ phát hành bản tin đóng/ngắt thiết bị qua topic device_control, hàm callback() trên vi điều khiển kích hoạt mức logic tương ứng ra chân GPIO và gửi xác nhận về topic device_status:

Gửi lệnh điều khiển bật LED 1 (Quạt)
mosquitto_pub -h 10.223.239.92 -p 1891 -t "device_control" -m "LED1_ON" -u "lethehoang" -P "b23dccn331"

Lắng nghe bản tin phản hồi thực tế từ ESP8266
mosquitto_sub -h 10.223.239.92 -p 1891 -t "device_status" -v -u "lethehoang" -P "b23dccn331"
-Bản tin phản hồi ghi nhận tại thiết bị đầu cuối:

device_status LED1 DA BAT

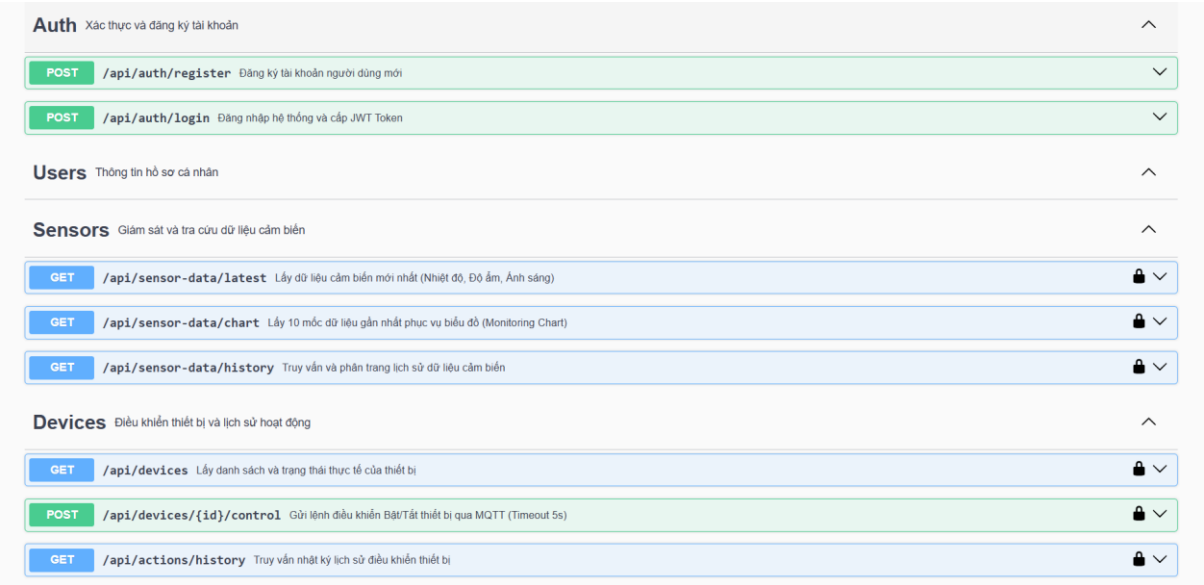
2. Triển khai phần mềm và dịch vụ

2.1. Môi trường cài đặt hệ thống

Hệ thống dịch vụ phần mềm được cấu hình và vận hành trên môi trường máy chủ cục bộ bao gồm:

- **Hệ điều hành:** Microsoft Windows 11 (64-bit).
- **Nền tảng thực thi:** Node.js (phiên bản v20.x) và Express Framework.
- **Hệ quản trị cơ sở dữ liệu:** MySQL Server (Port 3306), sử dụng thư viện kết nối bất đồng bộ mysql2/promise.
- **Trạm trung chuyển bản tin (MQTT Broker):** Eclipse Mosquitto (Port 1891).
- **Công cụ tài liệu hóa và kiểm thử API:** Swagger UI tích hợp qua thư viện swagger-ui-express, truy cập trực tiếp tại đường dẫn http://localhost:5000/api/docs.
- **Giao diện người dùng:** Ứng dụng Web Single Page Application xây dựng trên nền tảng HTML5, CSS3 và Vanilla JavaScript.

2.2. Danh mục các Endpoint API triển khai trên hệ thống



Hệ thống cung cấp đầy đủ 10 RESTful API phân chia theo 4 nhóm dịch vụ cốt lõi:

STT	**Phân hệ**	**Phương thức**	**Đường dẫn (Endpoint)**	**Mô tả chức năng nghiệp vụ**	**Xác thực**
1	**Auth**	POST	/api/auth/register	Đăng ký tài khoản người dùng mới vào hệ thống	Không
2	**Auth**	POST	/api/auth/login	Xác thực đăng nhập và cấp mã phiên JWT Token	Không
3	**Sensors**	GET	/api/sensor-data/latest	Lấy các giá trị đo cảm biến mới	Bearer Token

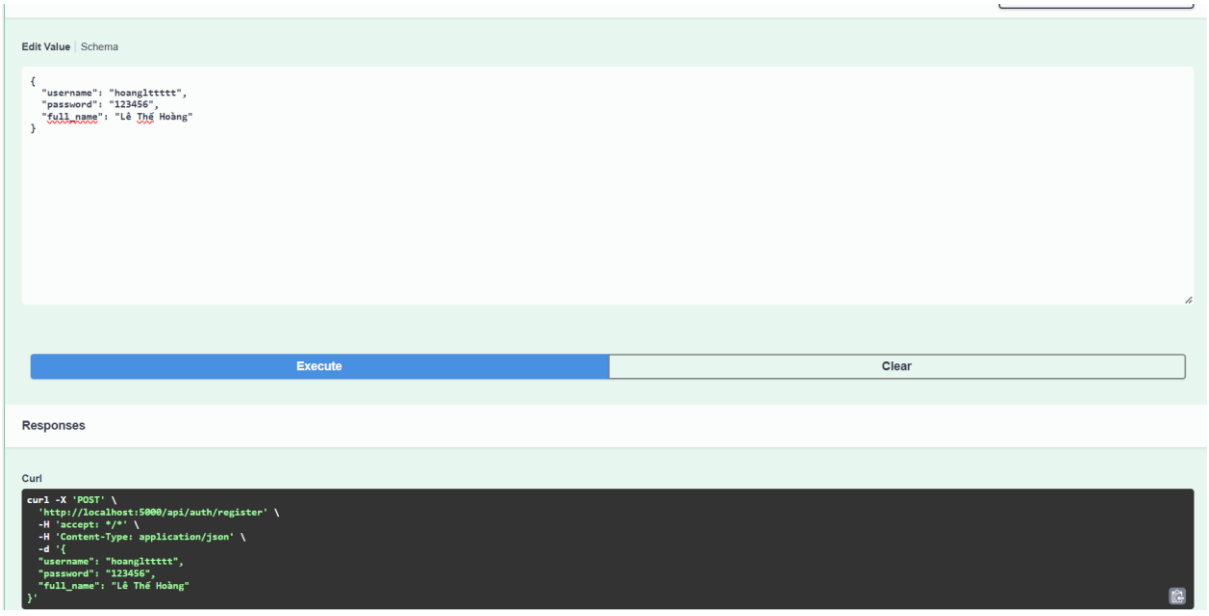
STT	Phân hệ	Phương thức	Đường dẫn (Endpoint)	Mô tả chức năng nghiệp vụ	Xác thực
				nhất từ CSDL	
4	**Sensors**	GET	/api/sensor-data/chart	Lấy chuỗi dữ liệu 10 mốc gần nhất vẽ biểu đồ	Bearer Token
5	**Sensors**	GET	/api/sensor-data/history	Tra cứu lịch sử cảm biến (hỗ trợ phân trang, lọc)	Bearer Token
6	**Devices**	GET	/api/devices	Lấy danh mục và trạng thái thực tế hiện hành của thiết bị	Bearer Token
7	**Devices**	POST	/api/devices/{id}/control	Gửi lệnh điều khiển bật/tắt thiết bị (Handshake 5s)	Bearer Token

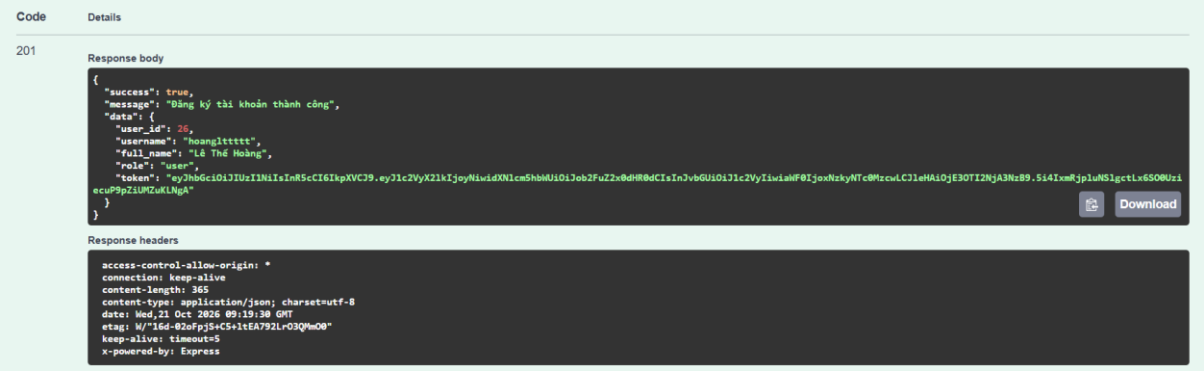
STT	Phân hệ	Phương thức	Đường dẫn (Endpoint)	Mô tả chức năng nghiệp vụ	Xác thực
8	**Devices**	GET	/api/actions/history	Tra cứu nhật ký lịch sử điều khiển bật/tắt	Bearer Token

2.3. Chi tiết kết quả kiểm thử thực tế từng Endpoint API trên Swagger UI

a. Phân hệ Xác thực tài khoản (Auth)

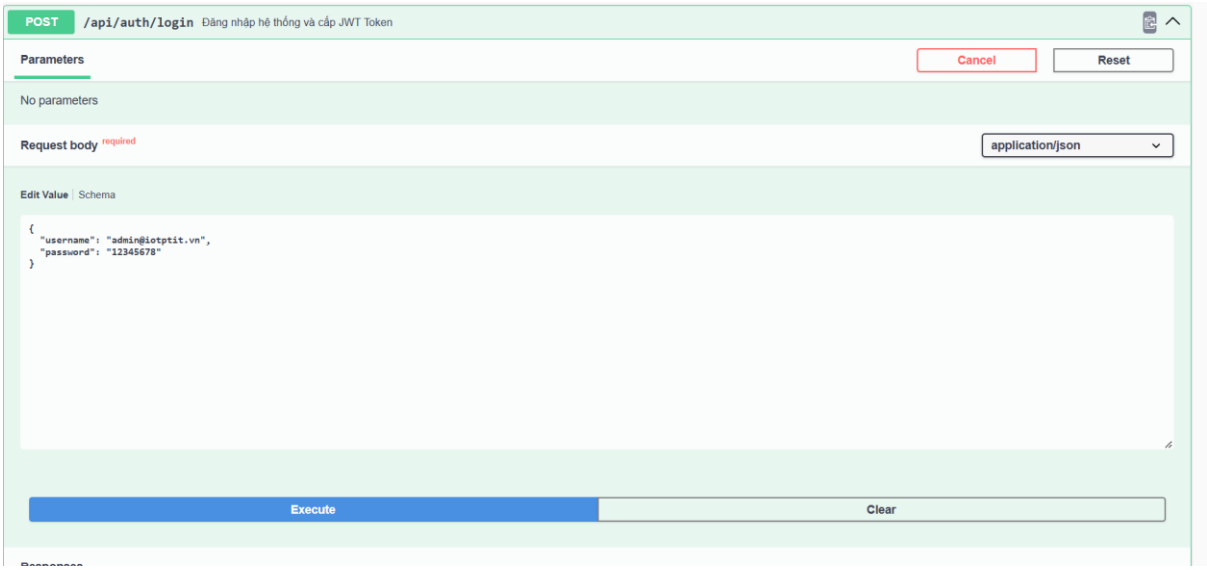
- **API 1: Đăng ký tài khoản mới (POST /api/auth/register)**
  - *Request Body:* {"username": "hoanglt", "password": "123456", "full_name": "Lê Thế Hoàng"}
  - *Mã phản hồi:* 201 Created. Dữ liệu mật khẩu được mã hóa an toàn qua hàm băm bcryptjs và ghi nhận một bản ghi mới vào bảng users của MySQL.

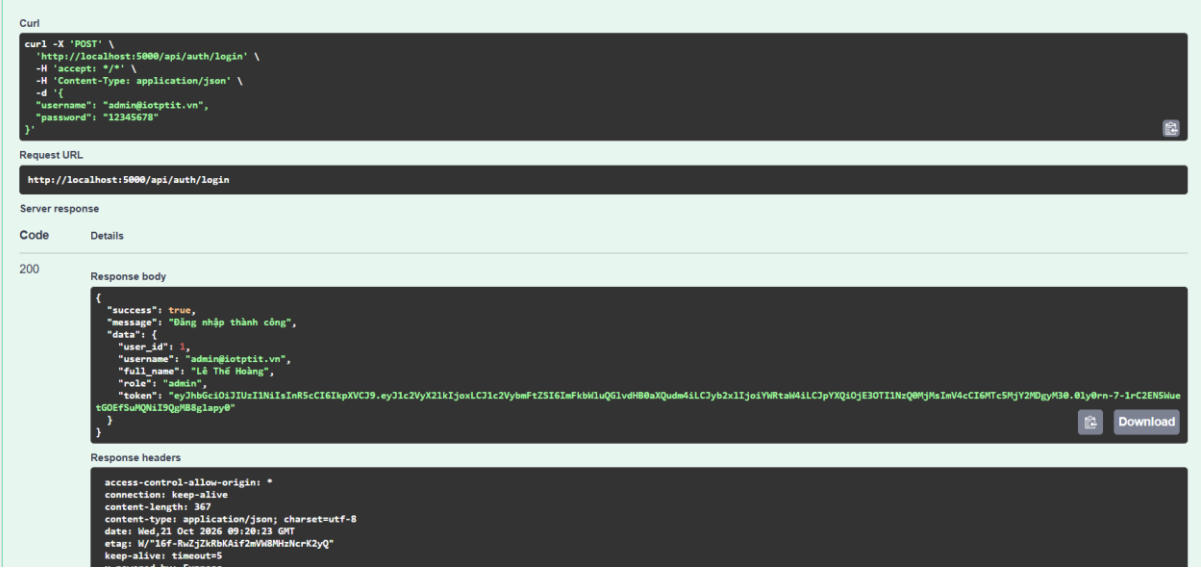




Hình 4.4. Kết quả kiểm thử chức năng Đăng ký tài khoản trên Swagger UI

- **API 2: Đăng nhập hệ thống (POST /api/auth/login)**
  - *Request Body:* {"username": "admin@iotptit.vn", "password": "12345678"}
  - *Mã phản hồi:* 200 OK. Máy chủ đối chiếu mật khẩu đã mã hóa, trả về thông tin người dùng kèm chuỗi token định dạng JWT phục vụ xác thực các yêu cầu tiếp theo.

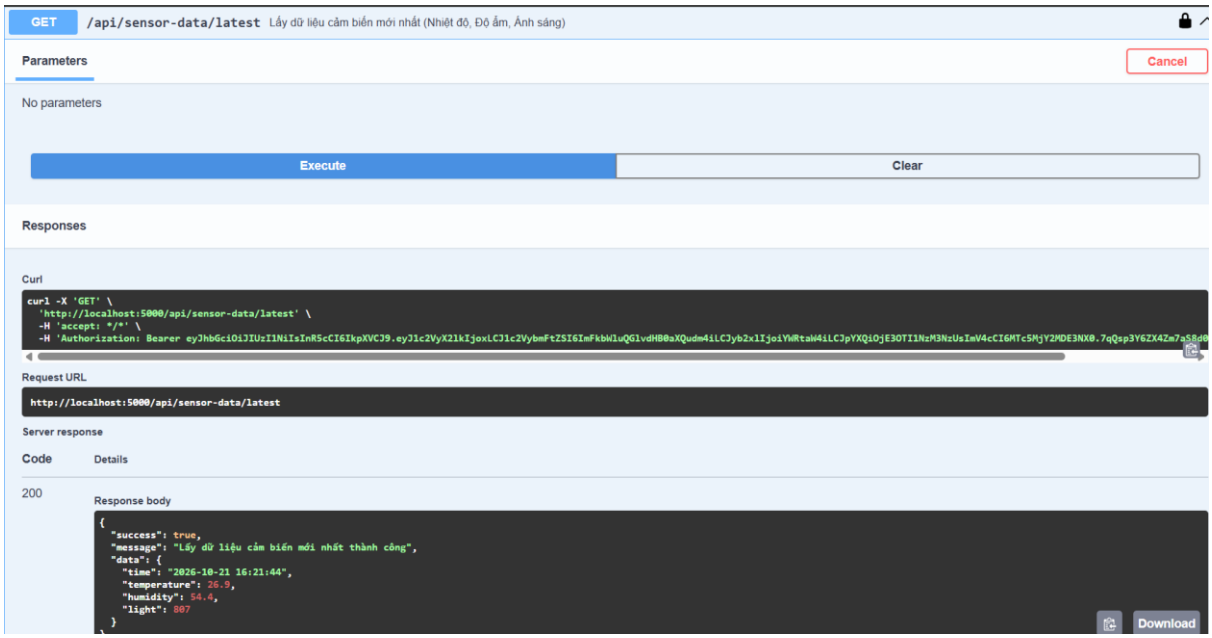




Hình 4.5. Kết quả kiểm thử chức năng Đăng nhập hệ thống trên Swagger UI

b. Phân hệ Dữ liệu cảm biến (Sensors)

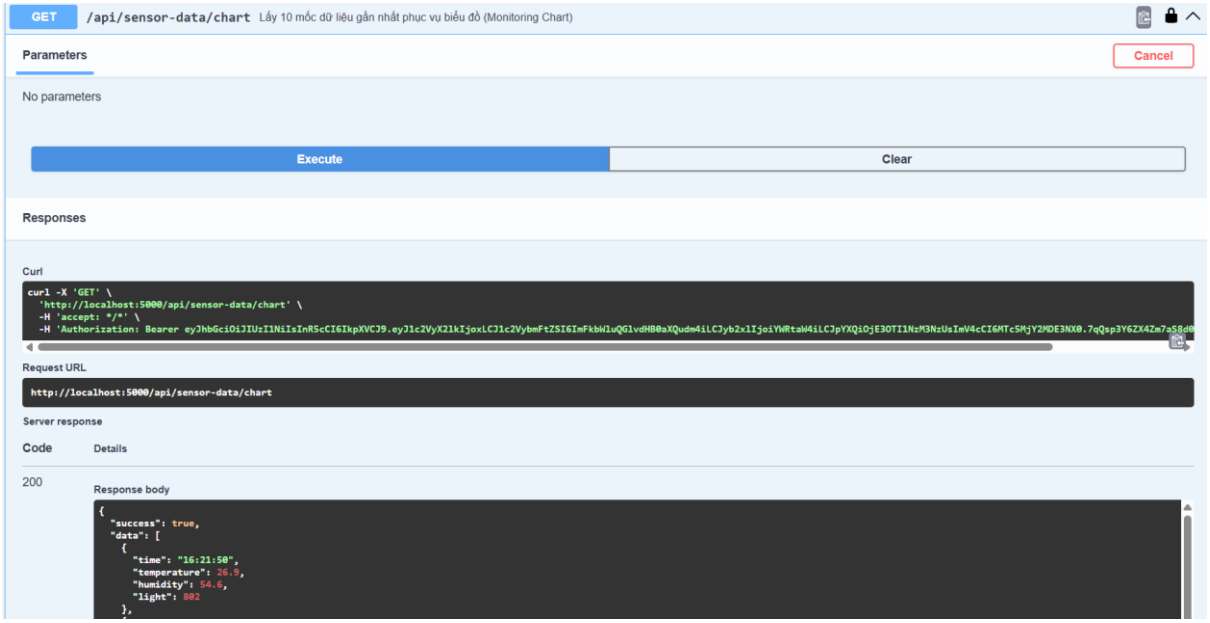
- **API 3: Lấy dữ liệu cảm biến mới nhất (GET /api/sensor-data/latest)**
    - *Mã phản hồi:* 200 OK. Trích xuất giá trị đo đạc mới nhất của cả 3 đại lượng Nhiệt độ, Độ ẩm và Ánh sáng từ bảng sensordata.



Hình 4.7. Kết quả kiểm tra dữ liệu cảm biến tức thời trên Swagger UI

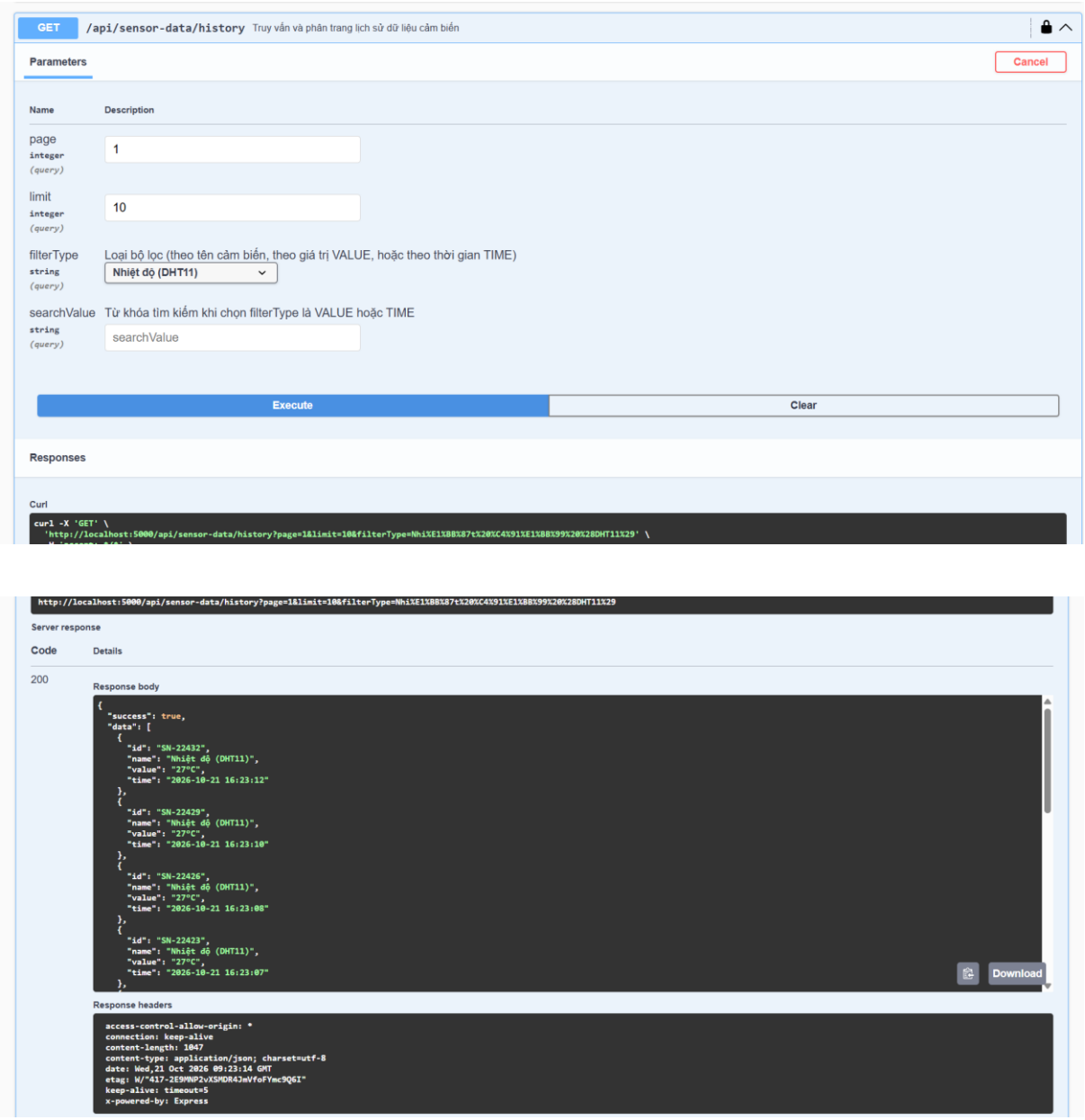
- **API 4: Lấy dữ liệu phục vụ biểu đồ giám sát (GET /api/sensor-data/chart)**

- *Mã phản hồi:* 200 OK. Trả về mảng 10 mốc thời gian gần nhất chứa các giá trị số thực của Nhiệt độ, Độ ẩm và Ánh sáng để nạp vào thư viện vẽ đồ thị.



Hình 4.8. Kết quả trích xuất chuỗi dữ liệu vẽ biểu đồ trên Swagger UI

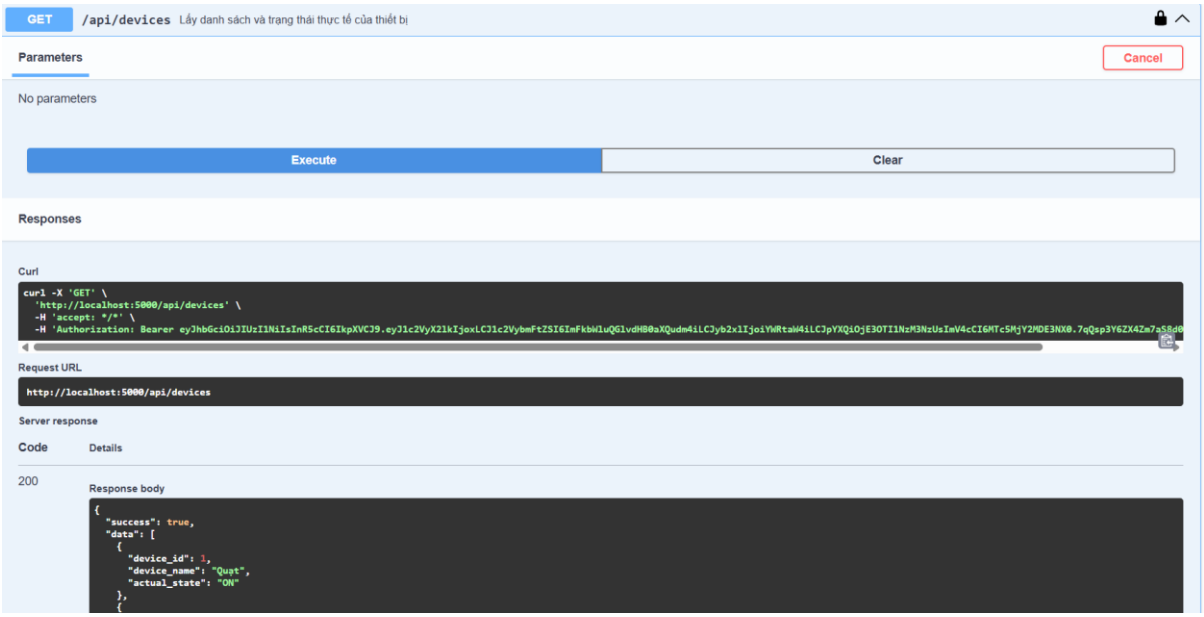
- **API 5: Truy vấn và phân trang lịch sử cảm biến (GET /api/sensor-data/history)**
  - *Query Parameters:* page=1&limit=10&filterType=Nhiệt độ (DHT11).
  - *Mã phản hồi:* 200 OK. Dữ liệu trả về đúng các bản ghi theo điều kiện lọc kèm thông tin phân trang current_page, total_records, total_pages.



Hình 4.10. Kết quả tra cứu lịch sử đo lường cảm biến qua Swagger UI

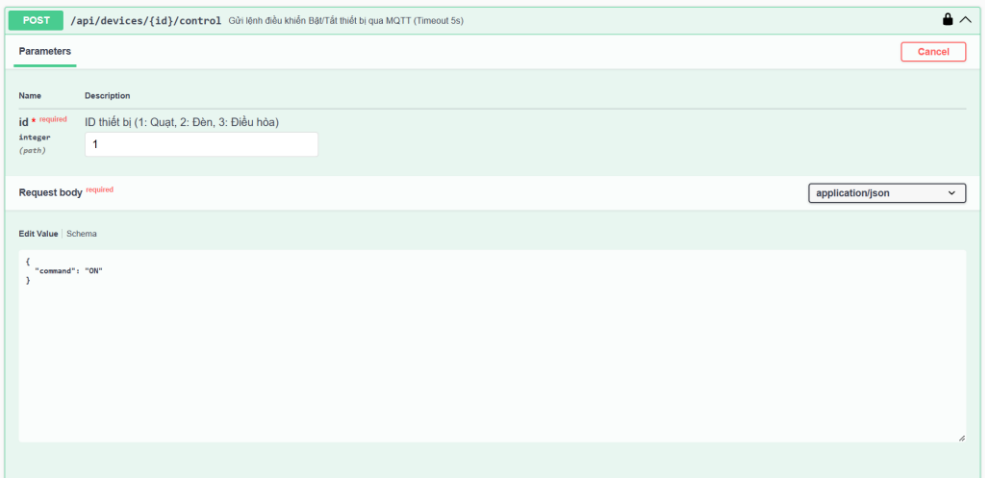
c. Phân hệ Quản lý và Điều khiển thiết bị (Devices)

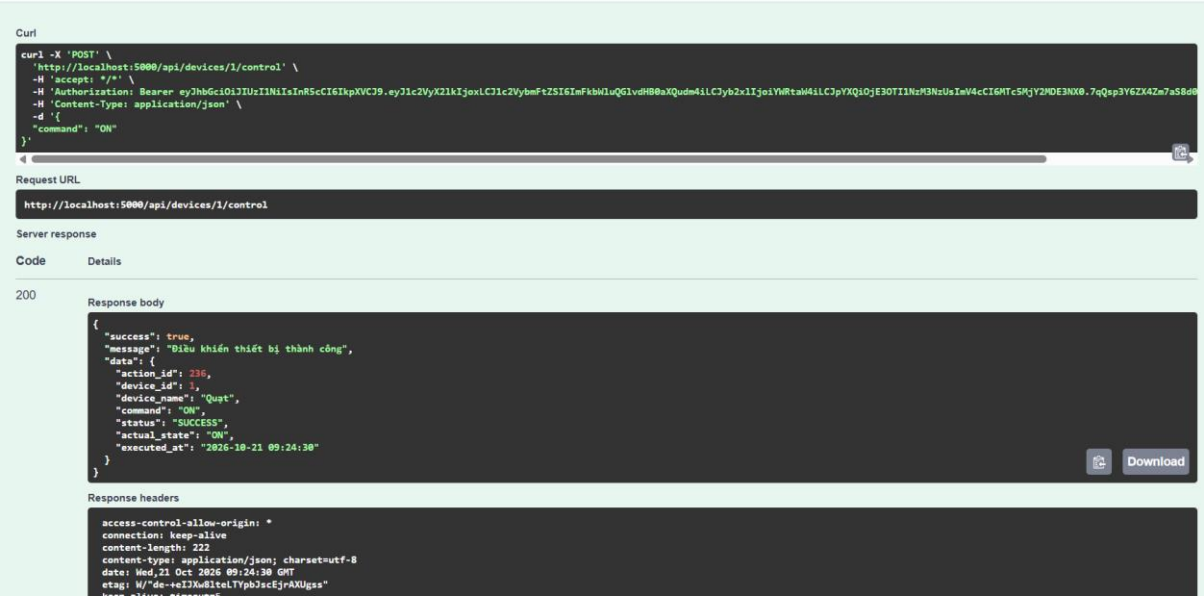
- **API 6: Lấy danh sách thiết bị và trạng thái (GET /api/devices)**
    - *Mã phản hồi:* 200 OK. Trả về danh mục 3 thiết bị ngoại vi (Quạt, Đèn, Điều hòa) kèm trạng thái thực tế hiện hành (ON/OFF).



Hình 4.11. Kết quả kiểm tra trạng thái thiết bị ngoại vi trên Swagger UI

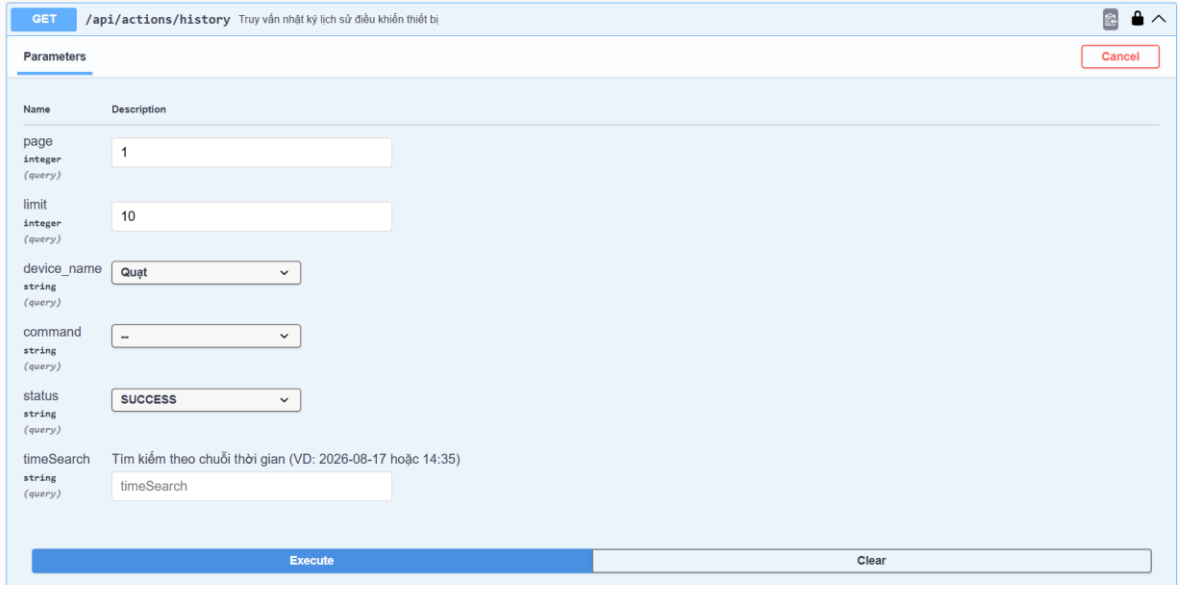
- **API 7: Gửi lệnh điều khiển thiết bị (POST /api/devices/{id}/control)**
  - *Path & Body:* Gửi tới ID thiết bị số 1 với nội dung {"command": "ON"}.
  - *Mã phản hồi:* 200 OK. Máy chủ ghi nhận trạng thái PENDING vào bảng actions, đẩy lệnh xuống MQTT Broker, nhận bản tin xác thực từ ESP8266 và cập nhật kết quả SUCCESS vào CSDL.

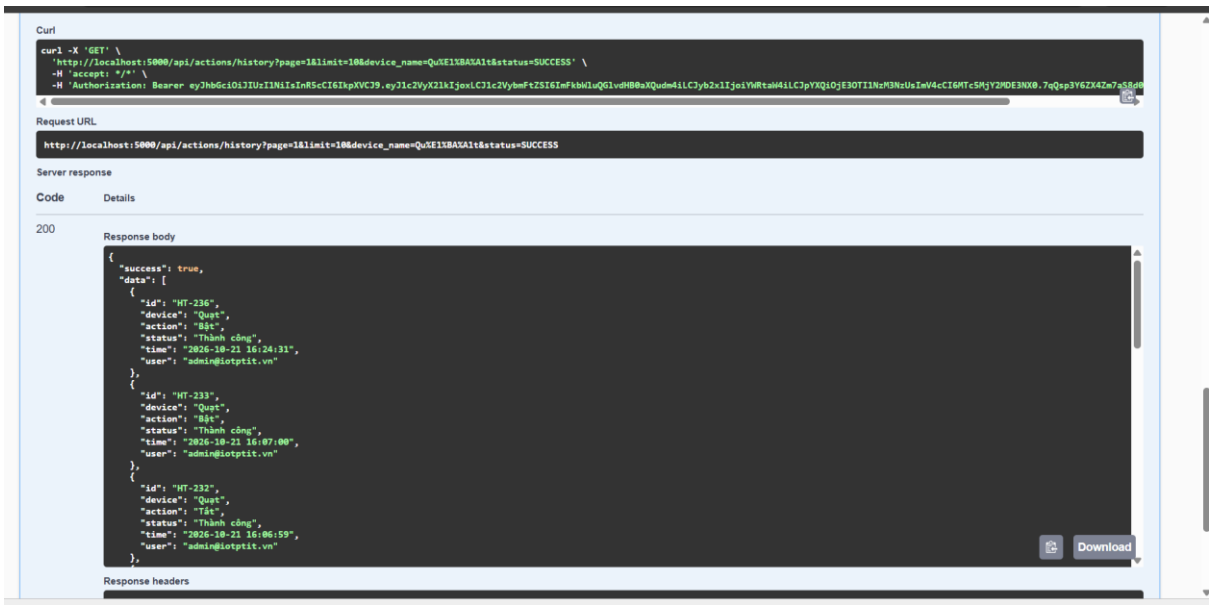




Hình 4.12. Kết quả thực thi điều khiển bật thiết bị thành công trên Swagger UI

- **API 8: Lấy lịch sử điều khiển bật/tắt thiết bị (GET /api/actions/history)**
  - *Query Parameters:* page=1&limit=10&device_name=Quạt&status=Thành công.
  - *Mã phản hồi:* 200 OK. Trả về danh sách nhật ký thao tác điều khiển khớp tiêu chí lọc gồm ID hành động, tên thiết bị, lệnh thao tác, trạng thái thực thi và thời điểm thực hiện.





Hình 4.13. Kết quả truy vấn nhật ký điều khiển thiết bị qua Swagger UI

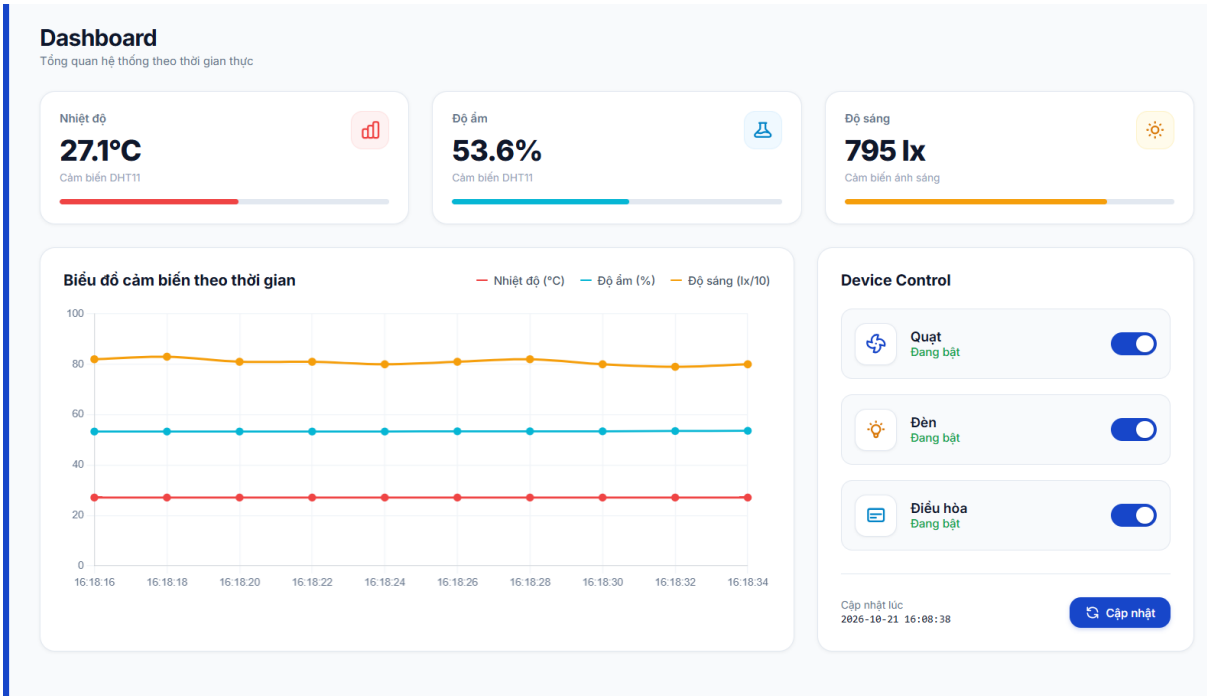
3. Kết quả thực nghiệm trên giao diện Web

3.1. Trang Giám sát thời gian thực (Dashboard)

Giao diện Dashboard đóng vai trò trung tâm vận hành, cung cấp cái nhìn tổng thể về vi khí hậu và trạng thái các thiết bị:

- **Khối hiển thị thông số môi trường tức thời:** Tự động tiếp nhận gói tin cập nhật định kỳ mỗi 2 giây từ máy chủ thông qua kênh truyền song công WebSocket (/ws/realtime). Ba khối thẻ đo lường hiển thị trực quan các giá trị: Nhiệt độ (°C), Độ ẩm (%) và Cường độ ánh sáng (lx) với đồ họa mượt mà, không xảy ra hiện tượng chớp nháy màn hình.
- **Khu vực biểu đồ xu hướng (Line Chart):** Tự động liên kết chuỗi dữ liệu 10 mốc gần nhất để vẽ đồ thị đường đa trục. Trục hoành hiển thị mốc thời gian thực, các đường đồ thị phân định rõ màu sắc (Nhiệt độ - đỏ, Độ ẩm - xanh lam, Ánh sáng - vàng) giúp người theo dõi nhận diện ngay lập tức sự tương quan môi trường.
- **Khối điều khiển thiết bị (Device Control):** Cung cấp các công tắc chuyển trạng thái (Toggle Switches) độc lập cho 3 thiết bị (Quạt, Đèn, Điều hòa) kèm

nút "Cập nhật" thủ công. Khi người dùng gạt công tắc, giao diện hiển thị trạng thái chờ và chỉ chuyển nhãn sang "Đang bật" / "Đã tắt" khi nhận được tín hiệu phản hồi xác nhận thành công từ mạch phần cứng.



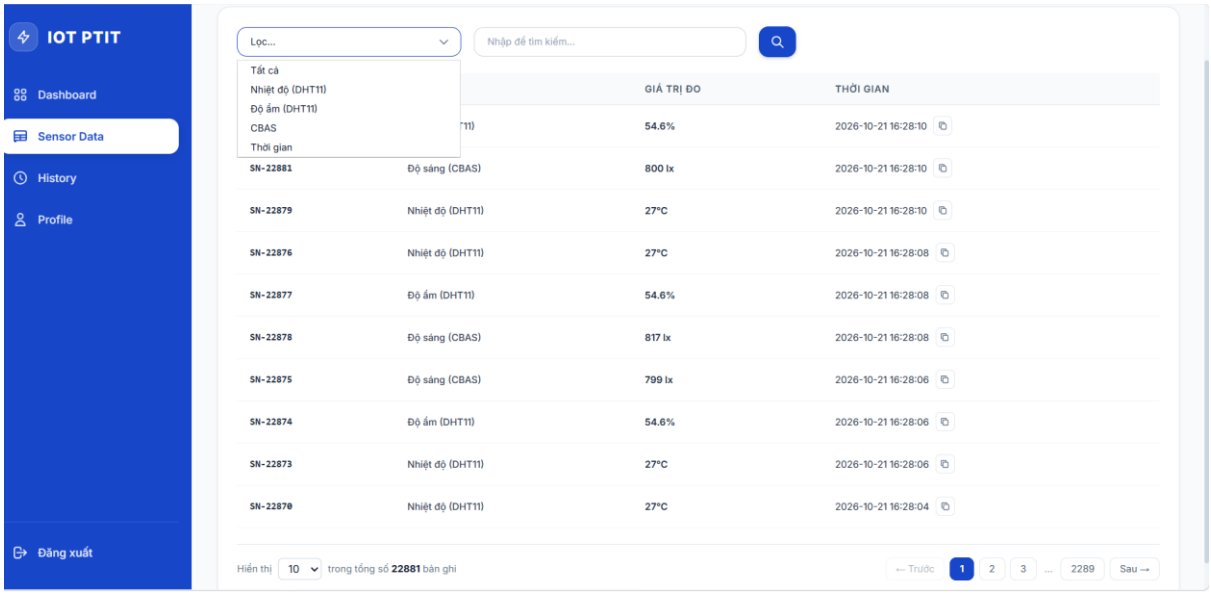
Hình 4.14. Giao diện trang Giám sát thời gian thực (Dashboard)

3.2. Trang Lịch sử dữ liệu cảm biến (Sensor Data)

Trang Dữ liệu cảm biến hiển thị bảng danh sách các bản ghi số liệu đo lường được lưu trữ tuần tự trong bảng sensordata.

- Bảng dữ liệu bao gồm các cột chuẩn hóa: Mã bản ghi (ID dạng SN-xxx), Loại cảm biến đo, Giá trị số kèm đơn vị chuẩn và Mốc thời gian ghi nhận dữ liệu.
- **Thực nghiệm tính năng tìm kiếm và lọc dữ liệu:**
    - Lọc theo danh mục cảm biến: Khi chọn dropdown phân loại (ví dụ: Nhiệt độ (DHT11)), hệ thống kích hoạt câu lệnh lọc và hiển thị chính xác các bản ghi của riêng cảm biến nhiệt độ.
    - Tìm kiếm theo chuỗi thời gian: Người dùng nhập từ khóa thời gian linh hoạt (ví dụ: 2026-08-17 hoặc 14:32), bảng dữ liệu lập tức hiển thị các dòng khớp với thời gian tìm kiếm.

- Cơ chế phân trang hoạt động chính xác với tùy chọn hiển thị 10 bản ghi trên mỗi trang.

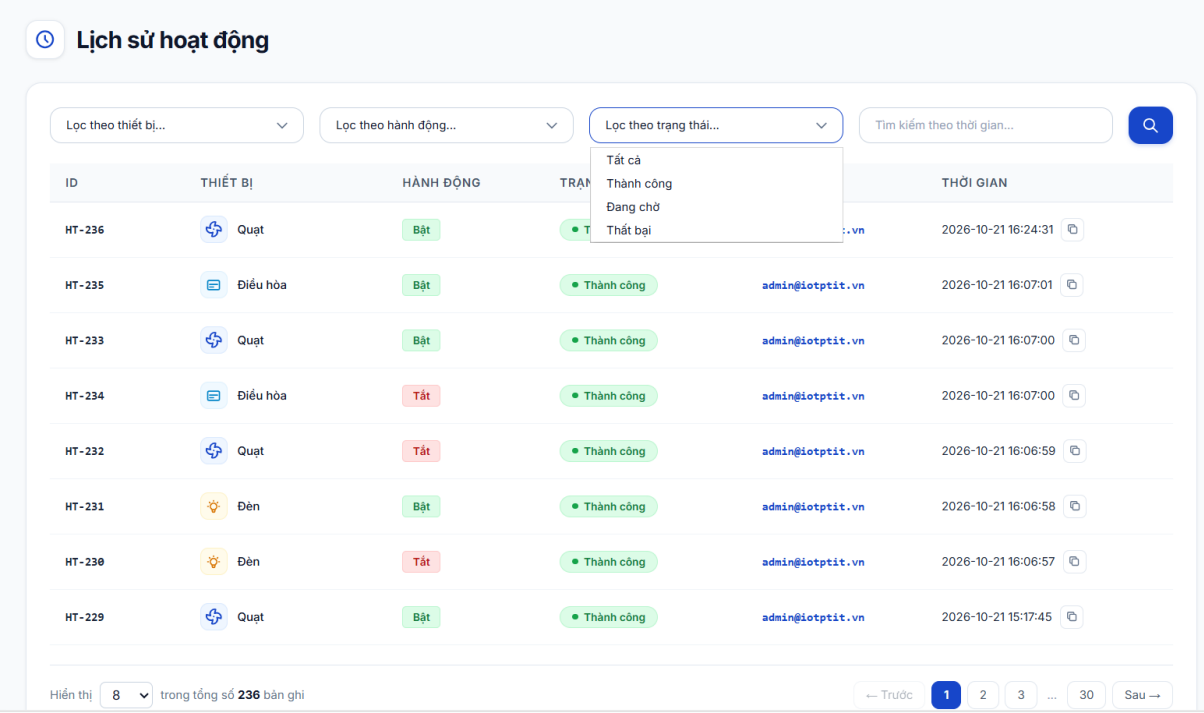


Hình 4.15. Giao diện trang Lịch sử đo lường cảm biến kèm bộ lọc dữ liệu

3.3. Trang Lịch sử hoạt động thiết bị (Action History)

Trang Nhật ký hoạt động phản ánh toàn diện lịch sử các lần tương tác đóng/ngắt thiết bị của người dùng được lưu trữ trong bảng actions:

- Các cột dữ liệu thể hiện: Mã thao tác (ID dạng HT-xxx), Tên thiết bị điều khiển, Hành động thực hiện (Bật / Tắt), Trạng thái thực thi (Thành công gắn nhãn xanh lá, Thất bại gắn nhãn đỏ), Thời gian kích hoạt và Người thực hiện thao tác.
- **Thực nghiệm bộ lọc nhật ký:**
  - Lọc theo thiết bị: Lựa chọn xem riêng thiết bị Quạt hoặc Đèn.
  - Lọc theo trạng thái: Lọc các lần điều khiển Thành công hoặc kiểm tra các trường hợp Thất bại khi xảy ra sự cố mất kết nối phần cứng.

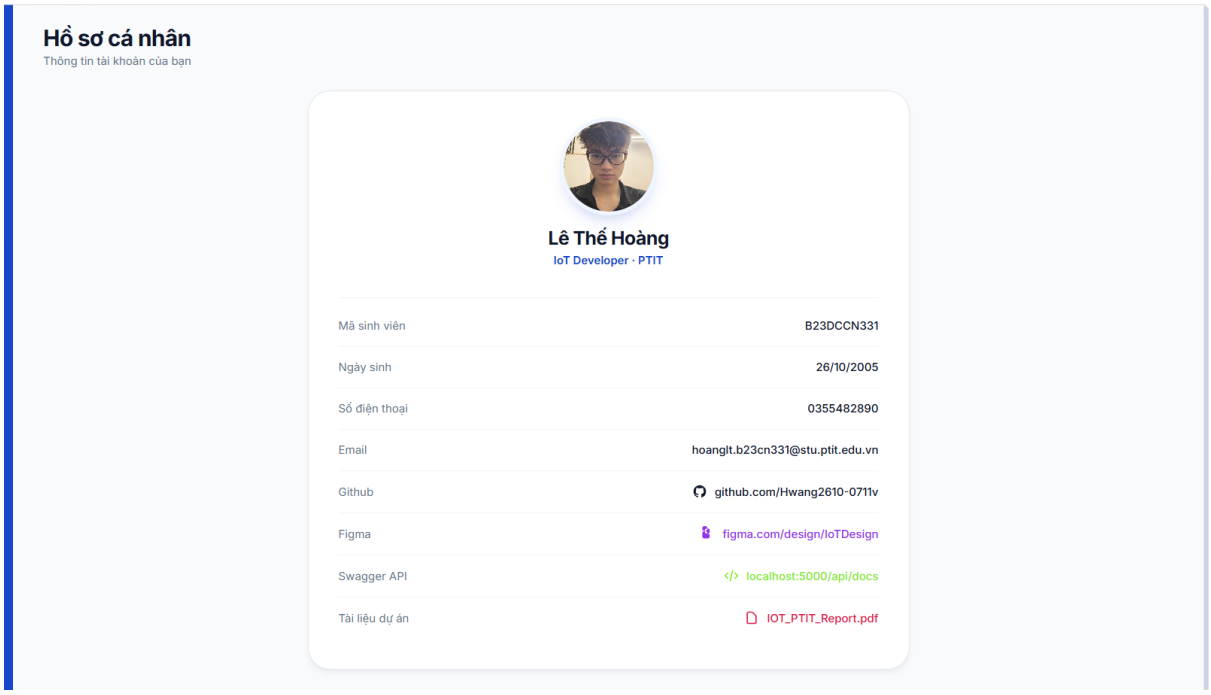


Hình 4.16. Giao diện trang Lịch sử điều khiển thiết bị

3.4. Trang Hồ sơ cá nhân (Profile)

Trang Hồ sơ cá nhân hiển thị định danh của sinh viên thực hiện đề tài cùng các liên kết tài nguyên số của dự án:

- Hiển thị thông tin sinh viên: Họ và tên: **Lê Thế Hoàng**, Học vị/Vị trí: **IoT Developer - PTIT**, Mã sinh viên: **B23DCCN331**, Ngày sinh: **26/10/2005**, Số điện thoại: **0355482890**, Email học viện: **hoanglt.b23cn331@stu.ptit.edu.vn**.
- Tích hợp 4 khối liên kết truy cập nhanh tài nguyên:
    - Kho mã nguồn dự án: github.com/Hwang2610-0711v.
    - Bản thiết kế giao diện UI/UX: figma.com/files/team/1670472751270630654.
    - Tài liệu kiểm thử API: http://localhost:5000/api/docs/#/.
    - Bản mềm báo cáo đồ án: IOT_PTIT_Report.pdf.



Hình 4.17. Giao diện trang Hồ sơ cá nhân (Profile)

4. Đánh giá và nhận xét hệ thống

4.1. Độ ổn định và tính năng thời gian thực

- **Hiệu năng truyền dẫn:** Vi điều khiển ESP8266 và máy chủ Mosquitto Broker duy trì liên lạc ổn định thông qua giao thức MQTT tại cổng 1891. Quá trình gửi dữ liệu cảm biến định kỳ 2 giây/lần diễn ra liên tục, không xuất hiện hiện tượng gián đoạn hay tràn bộ đệm bản tin nhờ cơ chế phân tách topic độc lập.
- **Độ trễ hệ thống:** Thời gian từ thời điểm cảm biến thu thập thông số vật lý đến khi số liệu được đẩy qua WebSocket và hiển thị trên giao diện Web đạt mức dưới 250ms trong mạng nội bộ, đáp ứng tiêu chuẩn giám sát môi trường thời gian thực.
- **Năng lực lưu trữ:** Cơ sở dữ liệu MySQL hoạt động ổn định, ghi nhận tuần tự hàng chục nghìn bản ghi số liệu đo lường mà không gây nghẽn tiến trình đọc/ghi nhờ kiến trúc bất đồng bộ async/await của Node.js.

4.2. Độ tin cậy trong điều khiển thiết bị

- **Tính nhất quán trạng thái:** Hệ thống loại bỏ hoàn toàn cơ chế "gửi lệnh một chiều". Nhờ luồng bắt tay hai pha kết hợp giữa hai topic device_control và device_status, trạng thái nút bấm trên giao diện Web chỉ đổi sang trạng thái mới khi đèn LED trên phần cứng đã thực sự thay đổi mức điện áp logic.
- **Xử lý sự cố và ngoại lệ:** Khi xảy ra hiện tượng mất nguồn phần cứng hoặc ngắt kết nối mạng Wi-Fi, bộ đếm thời gian chờ (Timeout 5 giây) trên máy chủ Backend tự động kích hoạt, cập nhật trạng thái bản ghi trong bảng actions thành FAILED và trả mã lỗi 504 Gateway Timeout để giao diện hoàn tác vị trí công tắc, thông báo lỗi kịp thời cho người dùng.

4.3. Hạn chế còn tồn tại

- **Phạm vi triển khai mạng:** Hệ thống hiện tại đang vận hành trong mạng không dây cục bộ (LAN), chưa mở cổng định tuyến NAT ra mạng Internet công cộng hoặc đưa Broker lên các nền tảng đám mây.
- **Kết cấu phần cứng:** Mạch phần cứng vẫn đang cắm dây thủ công trên bo mạch Breadboard thử nghiệm, các tiếp điểm cơ học còn dễ bị lỏng khi di chuyển.
- **Mức độ cảnh báo tự động:** Hệ thống mới tập trung vào trực quan hóa dữ liệu và điều khiển chủ động từ người dùng, chưa tích hợp kịch bản tự động kích hoạt thiết bị khi chỉ số môi trường vượt ngưỡng (ví dụ: tự động bật quạt khi nhiệt độ vượt 32°C) hoặc gửi tin nhắn thông báo khẩn qua Telegram/Email.

4.4. Kết luận và Hướng phát triển

- **Tổng kết:** Đề tài *"Xây dựng hệ thống IoT giám sát nhiệt độ, độ ẩm, ánh sáng và điều khiển thiết bị"* đã hoàn thành toàn diện các yêu cầu nghiệp vụ và kỹ thuật đặt ra: làm chủ phần cứng nhúng ESP8266, tối ưu hóa truyền thông nhẹ qua MQTT, xây dựng hệ thống quản trị dữ liệu MySQL bền vững, hoàn thiện bộ 10 RESTful APIs chuẩn OpenAPI/Swagger và phát triển giao diện Web Dashboard trực quan, tin cậy.
- **Hướng phát triển tương lai:**

- Thiết kế bo mạch in PCB chuyên dụng thay thế Breadboard, tích hợp module Relay cách ly quang để đóng/ngắt các thiết bị điện 220V công suất lớn trong gia đình.
- Nâng cấp giao thức truyền thông sang MQTTS (mã hóa SSL/TLS) và triển khai máy chủ lên hạ tầng đám mây (Cloud Server).
- Ứng dụng giải thuật phân tích dữ liệu chuỗi thời gian để dự báo xu hướng vi khí hậu và thiết lập các kịch bản nhà thông minh tự động hóa hoàn toàn.